COMPLETE LEARNING GUIDE · 2026 EDITION

Tailwind CSS

From Fundamentals to Advanced — utility-first styling explained in simple, Tamil-friendly language with real-world examples.

இது ஒரு முழுமையான Tailwind CSS கற்றல் கையேடு — எளிய விளக்கங்கள், practice tasks, interview questions மற்றும் mini projects உடன்.

13 Chapters | 50 Interview Questions | 4 Mini Projects | Tamil-Friendly Explanations

AUTHOR

Dhamodharan

Tailwind CSS v4 Focused
Practice Tasks & Cheat Sheets Included

PREFACE

About This Ebook

How this guide is organized, who it is for, and how to get the most out of it.

Indha ebook, Tailwind CSS-ai zero-la irundhu advanced level varaikkum kathukka vendi oru complete guide. Basics, layout, responsive design, typography, animations, advanced techniques, forms, interview questions and mini projects — ellame ore organised PDF-la kudukkapattirukku.

Who Is This Book For?

- **Beginners** — HTML and CSS basics theriyum, aana Tailwind CSS new.
- **Students & Job Seekers** — interview preparation-ku ready-ah irukkura 50 Q&A and cheat sheets.
- **Working Developers** — quick reference-ah v4 syntax, advanced patterns and production tips.

How to Use This Book

1. Ovvoru chapter-um order-ah padikkavum — basics strong-ah aana piragu advanced topics easy-ah puriyum.
2. Code examples-ai copy-paste pannaama, unga own project-la type panni practice pannavum.
3. Ovvoru chapter-oda **Practice Task** box-ai complete pannavum — adhu dhaan real learning.
4. Revision-ku chapter-oda **Cheat Sheet** tables use pannavum.
5. Kadaisi-yila **Mini Projects** build pannavum — ellaa skills-um combine aagum.

Conventions Used in This Book

Element	Meaning
Dark code blocks	HTML / CSS examples that you can try in your project.
Note boxes (blue)	Important reminders, version differences and gotchas.
Tip boxes (green)	Helpful ideas and best practices.
Practice Task boxes (amber)	Hands-on exercises to build real skill.
Tables	Quick reference of classes and their meanings.

> **NOTE**
> Indha ebook-la Tailwind CSS **v4 syntax** primary-ah use pannappattirukku. Version differences (v3 vs v4) thevaiyana idathula mention pannirukkom. Latest features-ku always official docs paarunga: tailwindcss.com/docs.

What You Will Learn

- Utility-first thinking — normal CSS vs Tailwind CSS.
- Core utilities: layout, display, spacing, sizing, colors, borders and shadows.
- Flexbox, Grid and responsive design (mobile-first).
- Backgrounds, gradients, visual effects, typography and text control.
- States and variants: hover, focus, group, peer and dark mode.
- Transitions, transforms and animations.
- Advanced topics: arbitrary values, @theme tokens, custom utilities and performance.
- Forms, validation states and interactive components like dropdowns and modals.

CONTENTS

Table of Contents

Thirteen chapters — from fundamentals to interview-ready knowledge.

01 **Introduction to Tailwind CSS** — 4
What it is, utility-first vs normal CSS, advantages, common classes and how it works.

02 **Connecting Tailwind CSS with HTML and React** — 6
Play CDN setup for HTML projects, Vite + Tailwind v4 setup for React, and a quick comparison.

03 **Core Utility Classes** — 9
Layout, display, position, overflow, spacing, sizing, typography, colors, borders and shadows.

04 **Flexbox and Grid in Tailwind CSS** — 10
One-dimensional vs two-dimensional layouts, alignment classes, and responsive flex & grid.

05 **Responsive Design in Tailwind CSS** — 14
Breakpoints, mobile-first approach, responsive width, flex, grid, spacing and visibility.

06 **Backgrounds and Visual Effects** — 17
Background colors, gradients, images, borders, radius, shadows, opacity, blur and backdrop effects.

07 **Typography and Text** — 22
Fonts, sizes, weights, line height, letter spacing, alignment, truncation, lists and line clamping.

08 **States and Variants** — 30
hover, focus, active, disabled, first/last/odd/even, group, peer, dark mode and form states.

09 **Transitions and Animations** — 36
Transitions, transforms, built-in animations, custom keyframes, loaders and reduced motion.

10 **Advanced Tailwind CSS** — 43
Arbitrary values, @theme tokens, custom utilities and variants, components, dark mode and performance.

11 **Forms and Interactivity** — 52
Inputs, validation states, accent & caret, cursors, scrolling, sticky headers, dropdowns and modals.

12 **50 Interview Questions and Answers** — 61
Complete revision across all thirteen topics, from beginner to advanced.

13 **Mini Projects — Combine Your Skills** — 69
Four hands-on projects to combine everything you learned.

TIP

Interview preparation-ku pokiravangal, Chapter 12 (50 Questions) mattum illaama, ovvoru chapter-oda cheat sheet tables-um revise pannunga — adhu dhaan fastest revision method.

CHAPTER 01

Introduction to Tailwind CSS

Tailwind CSS na enna, eppadi work aagum, and why it is different from normal CSS.

1. Tailwind CSS na enna?

Tailwind CSS enbadhu oru **Utility-First CSS Framework**. Idha use panni namma HTML elements-ku direct-ah predefined CSS classes apply panni, website-ku styling kodukkalaam.

Normal CSS-la namma thaniya CSS properties ezhuthuvom. Aana Tailwind CSS-la already irukkura utility classes-ai use pannalaam.

2. Normal CSS vs Tailwind CSS

NORMAL CSS EXAMPLE

```
<h1>Hello World</h1>

<style>
h1 {
  color: blue;
  font-size: 30px;
  text-align: center;
}
</style>
```

TAILWIND CSS EXAMPLE

```
<h1 class="text-blue-500 text-3xl text-center">
  Hello World
</h1>
```

Explanation:

- `text-blue-500` → Text-ku blue color kodukkum.
- `text-3xl` → Text size-ah perusa maathum.
- `text-center` → Text-ah center-la align pannum.

Inga separate-ah CSS ezhuthaama, HTML-la Tailwind classes use panni styling pannirukkom.

3. Tailwind CSS-oda Main Advantages

Advantage	Explanation
1. Faster Development	Kammi time-la attractive website designs create pannalaam.
2. Utility-First Approach	Ready-made classes use panni styling pannalaam.
3. Responsive Design	Mobile, tablet, desktop-ku suitable-ah website create pannalaam.
4. Easy Customization	Colors, spacing, font sizes, layouts pondravatthai easy-ah customize pannalaam.

4. Common Tailwind CSS Classes

Class	Meaning
`text-red-500`	Text color red
`bg-blue-500`	Background blue
`text-xl`	Text size increase
`font-bold`	Bold text
`p-4`	Ella sides-layum padding
`m-4`	Ella sides-layum margin
`flex`	Flexbox layout
`grid`	Grid layout
`rounded-lg`	Rounded corners
`shadow-lg`	Large shadow

5. Small Practical Example

```
<div class="bg-white p-6 rounded-lg shadow-lg">
  <h1 class="text-2xl font-bold text-blue-600">
    Welcome to Tailwind CSS
  </h1>

  <p class="text-gray-600 mt-2">
    Learn Tailwind CSS step by step!
  </p>

  <button class="bg-blue-500 text-white px-4 py-2 rounded-lg mt-4">
    Get Started
  </button>
</div>
```

Indha code enna pannum?

- `bg-white` → White background.
- `p-6` → Content suththi padding.
- `rounded-lg` → Rounded corners.
- `shadow-lg` → Shadow effect.
- `mt-2` → Top margin.
- `px-4 py-2` → Button-ku horizontal and vertical padding.

> **NOTE**
> Indha example work aaganumna project-la **Tailwind CSS setup** pannirukkanum. (Vite / CLI / PostCSS — edho oru integration thevai.)

6. Tailwind CSS Eppadi Work Aagum?

HTML Elements	→	Tailwind Utility Classes	→	Styled Website

h1, p, button	text-xl, bg-blue-500, p-4	Color, spacing, layout and design

Tailwind build process use panni, HTML-la use pannirukkura classes-kku thevaiyana CSS generate pannappadum. Unused classes-in CSS build output-la varadhu.

7. Tailwind CSS Learn Panna Enna Therinjirukkanum?

- HTML basics
- CSS properties
- Box model
- Flexbox and Grid basics
- Responsive design basics

REMEMBER

Tailwind CSS enbadhu CSS-ku **replacement language illa**; CSS styling-ai utility classes moolama easy-ah apply panna help pannura framework. So CSS fundamentals theriyanum — adhu unga strength-ai double aakkum.

Official documentation: Tailwind CSS Docs

PRACTICE TASK

- Create an HTML page and connect Tailwind CSS.
- Build a simple card using utility classes without writing custom CSS.
- Create a heading, paragraph, button, and image using Tailwind.
- Compare the same component built with regular CSS and Tailwind CSS.

CHAPTER 02

Connecting Tailwind CSS with HTML and React

HTML project-la mattum illa, React project-la Tailwind CSS eppadi add pannanum — step-by-step.

Tailwind CSS-na oru utility-first CSS framework. Idha use panni namma HTML elements-ku direct-ah classes add panni styling pannalam. Separate-ah CSS code ezhuthuradhu reduce aagum. Example: `bg-blue-500`, `text-white`, `p-4`, `rounded-lg` madhiri classes use pannuvom.

1. HTML-la Tailwind CSS add panradhu

HTML project-la Tailwind CSS use panna rendu common methods irukku. Beginners-ku CDN method easy; proper projects-ku CLI or build setup use pannalaam.

Step 1: HTML file create pannunga

`index.html` nu oru file create pannunga.

Step 2: Tailwind Play CDN add pannunga

`<head>` section-kulla indha script add pannunga. Idhu quick testing and learning-ku suitable.

```
<script src="https://cdn.tailwindcss.com"></script>
```

FULL EXAMPLE

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <title>Tailwind Example</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>

<body class="bg-gray-100 p-8">

  <h1 class="text-3xl font-bold text-blue-600 p-4">
    Hello Tailwind CSS!
  </h1>

  <button class="bg-blue-500 text-white px-4 py-2 rounded-lg">
    Click Me
  </button>

</body>
</html>
```

Explanation (Tanglish):

- `bg-gray-100` — background light gray-ah irukkum.
- `text-3xl` — text size perusa irukkum.
- `font-bold` — text bold-ah irukkum.
- `text-blue-600` — text blue color-la irukkum.
- `p-4` — element-sutthi padding add aagum.
- `px-4 py-2` — horizontal and vertical padding.
- `rounded-lg` — button corners rounded-ah irukkum.

> **NOTE**
> Tailwind Play CDN development/testing-ku mattum. Production website-ku proper build setup use pannunga.

2. React-la Tailwind CSS add panradhu

React project-la proper-ah Tailwind CSS install panna Vite + Tailwind CSS v4 method use pannalaam.

Step 1: React project create pannunga

Already React project irundha indha step skip pannalaam. Terminal-la indha commands run pannunga:

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
```

- `npm create vite@latest` — Vite project create pannum.
- `my-react-app` — project name.
- `--template react` — React setup create pannum.
- `npm install` — required packages install pannum.

Step 2: Tailwind CSS install pannunga

Project folder-kulla indha command run pannunga:

```
npm install tailwindcss @tailwindcss/vite
```

Step 3: vite.config.js update pannunga

Project root-la irukkura `vite.config.js` file open panni, indha code use pannunga:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],
})
```

Explanation: `tailwindcss()` plugin Vite-oda connect aagi, Tailwind styles generate aaga help pannum.

Step 4: src/index.css update pannunga

Existing CSS file-la irukkura content-ai replace panni, indha line add pannunga:

```
@import "tailwindcss";
```

Idhu Tailwind CSS styles-ai project-kulla import pannum.

Step 5: src/App.jsx-la use pannunga

`src/App.jsx` file-ai indha code-poda replace pannunga:

```
function App() {
  return (
    <div>
      <h1 className="text-3xl font-bold text-blue-600">
        Welcome to React + Tailwind CSS!
      </h1>

      <p className="mt-3 text-gray-700">
        Tailwind CSS is working successfully.
      </p>

      <button className="mt-4 rounded-lg bg-blue-600 px-5 py-2 text-white hover:bg-blue-
700">
        Get Started
      </button>
    </div>
  )
}

export default App
```

Mukkiyamaana difference:

- HTML-la `class="..."` use pannuvom.
- React JSX-la `className="..."` use pannanum.

Step 6: CSS import check pannunga

`src/main.jsx` file-la `index.css` import aagirukkanum:

```
import './index.css'
```

Step 7: Project run pannunga

Terminal-la indha command run pannunga:

```
npm run dev
```

Terminal-la varra local URL-ai browser-la open pannunga. Ippo Tailwind styles work aaganum.

3. HTML vs React — quick comparison

Feature	HTML	React
Setup	CDN for quick testing	npm installation
Class attribute	`class`	`className`
CSS connection	HTML `<head>` or CSS file	CSS import in `main.jsx`
Example	`class="text-red-500"`	`className="text-red-500"`

LEARNING TIP

First HTML-la Tailwind classes practice pannunga. Adhukku appuram React-la className, components, responsive classes (`sm:` , `md:` , `lg:`) practice pannina easy-ah puriyum.

PRACTICE TASK

- HTML file-la Play CDN use panni oru simple page build pannunga.
- Vite + React project create panni Tailwind CSS-ai install pannunga.
- Oru component-la className use panni styling pannunga.
- HTML setup-um React setup-um compare pannunga.

CHAPTER 03

Core Utility Classes

Topic-wise quick reference of Tailwind-oda core utility classes.

Indha chapter-la layouts, display, spacing, sizing, typography, colors, borders and shadows-ku thevaiyana core utility classes-ai topic-wise-ah paarpom. Idhu oru quick reference — project-la work pannumbodhu thirumbi thirumbi use pannalaam.

Topic	Important concepts
Layout	`container` , `box-border` , `box-content` , `isolate`
Display	`block` , `inline` , `inline-block` , `flex` , `grid` , `hidden`
Position	`static` , `relative` , `absolute` , `fixed` , `sticky`
Float & clear	`float-left` , `float-right` , `clear-both`
Overflow	`overflow-auto` , `overflow-hidden` , `overflow-scroll`
Object fit	`object-cover` , `object-contain` , `object-fill`

Topic	Important concepts
Object position	`object-center`, `object-top`, `object-bottom`
Spacing	`p-*`, `px-*`, `py-*`, `m-*`, `mx-*`, `my-*`, `gap-*`
Sizing	`w-*`, `h-*`, `min-w-*`, `max-w-*`, `min-h-*`, `max-h-*`
Typography	`font-*`, `text-*`, `leading-*`, `tracking-*`
Colors	`text-*`, `bg-*`, `border-*`, `fill-*`
Borders	`border`, `rounded-*`, `ring-*`, `outline-*`
Shadows	`shadow-*`, `drop-shadow-*`

TIP

`p-*` maadhiri star syntax na — antha family-la pala values irukku. Example: `p-1`, `p-2`, `p-4`, `p-6`, `p-8` etc. Value perusa aana spacing-um perusa aagum.

PRACTICE TASK

- Create a profile card using margin, padding, width, and height utilities.
- Design a button using background colors, text colors, borders, and rounded corners.
- Create three cards with different border styles and shadows.
- Build a two-column layout using width and spacing utilities.
- Create a page with different font sizes, font weights, and text alignment.

CHAPTER 04

Flexbox and Grid in Tailwind CSS

Layout create panna rendu power tools — eppo edhu use pannanum-nu clear-ah purinjikonga.

Tailwind CSS-la Flexbox and Grid rendu perum website layout create panna use aagura utility classes. Aana rendu perukkum different purpose irukku.

- **Flexbox** → Items-ai row illa column-la arrange panna useful.
- **Grid** → Items-ai rows and columns format-la arrange panna useful.

1. Flexbox in Tailwind CSS

Flexbox use panni elements-ai horizontal-ah, vertical-ah arrange pannalaam. Items-kku naduvula space kodukkalaam, center align pannalaam.

Important Flexbox Classes

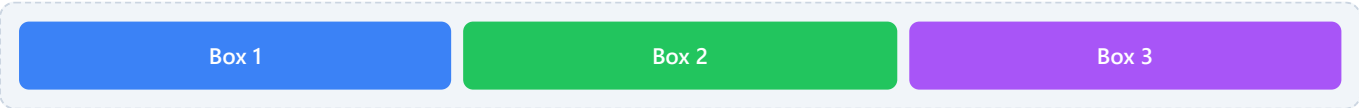
Tailwind class	Explanation
`flex`	Flexbox layout enable pannum
`inline-flex`	Inline flex container create pannum
`flex-row`	Items horizontal-ah arrange aagum
`flex-row-reverse`	Horizontal order reverse aagum
`flex-col`	Items vertical-ah arrange aagum

Tailwind class	Explanation
`flex-col-reverse`	Vertical order reverse aagum
`flex-wrap`	Space illana next line-ku pogum
`flex-nowrap`	Items ore line-la irukkum
`flex-1`	Available space-ai occupy panna help pannum
`flex-auto`	Flex item grow and shrink aagum; basis auto
`flex-none`	Flex item grow/shrink aagadhu
`flex-shrink-0`	Item shrink aaguradhai prevent pannum

Alignment Classes

Tailwind class	Explanation
`justify-start`	Main axis start-la align pannum
`justify-center`	Main axis center-la align pannum
`justify-end`	Main axis end-la align pannum
`justify-between`	Items-kku naduvula space
`justify-around`	Items suththi space
`justify-evenly`	Equal spacing
`items-start`	Cross axis start
`items-center`	Cross axis center
`items-end`	Cross axis end
`items-stretch`	Available cross-axis space-ku stretch pannum
`gap-4`	Items-kku naduvula gap

Flexbox Visual Example



FLEXBOX CODE EXAMPLE

```
<div class="flex flex-row justify-between items-center gap-4">
  <div class="bg-blue-500 p-4 text-white">Box 1</div>
  <div class="bg-green-500 p-4 text-white">Box 2</div>
  <div class="bg-purple-500 p-4 text-white">Box 3</div>
</div>
```

Explanation:

- `flex` → Flexbox enable pannum.
- `flex-row` → Horizontal layout.
- `justify-between` → Items-kku naduvula space distribute pannum.
- `items-center` → Cross axis-la center align pannum.

- `gap-4` → Items-kku idayila gap set pannum.

> **NOTE**
> `justify-between` use pannumbodhu available space distribute aagum. `gap-4` additional minimum gap-ai set pannum.

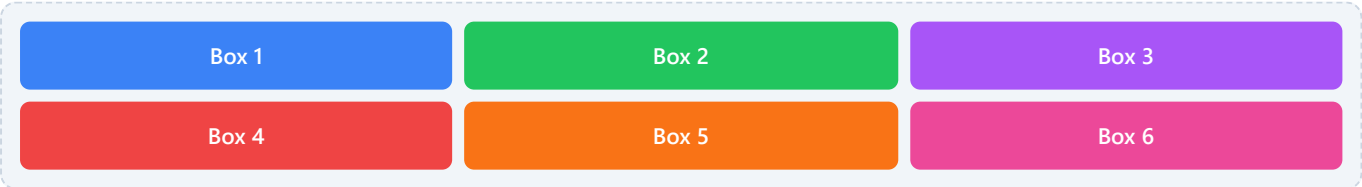
2. Grid in Tailwind CSS

CSS Grid use panni rows and columns-la elements-ai arrange pannalaam. Product cards, portfolio skills, image galleries, dashboards pondra layouts-ku romba useful.

Important Grid Classes

Tailwind class	Explanation
`grid`	Grid layout enable pannum
`grid-cols-1`	One column
`grid-cols-2`	Two columns
`grid-cols-3`	Three columns
`grid-cols-4`	Four columns
`grid-rows-2`	Two explicit rows
`col-span-2`	Two columns occupy pannum
`row-span-2`	Two rows occupy pannum
`grid-flow-row`	Items row order-la place aagum
`grid-flow-col`	Items column direction-la place aagum
`gap-4`	Rows and columns-kku gap
`gap-x-4`	Column gap
`gap-y-4`	Row gap

Grid Visual Example



GRID CODE EXAMPLE

```
<div class="grid grid-cols-3 gap-4">
  <div class="bg-blue-500 p-4 text-white">Box 1</div>
  <div class="bg-green-500 p-4 text-white">Box 2</div>
  <div class="bg-purple-500 p-4 text-white">Box 3</div>
  <div class="bg-red-500 p-4 text-white">Box 4</div>
  <div class="bg-orange-500 p-4 text-white">Box 5</div>
  <div class="bg-pink-500 p-4 text-white">Box 6</div>
</div>
```

Explanation:

- `grid` → Grid layout enable pannum.
- `grid-cols-3` → Three equal-width columns create pannum.
- `gap-4` → Grid items-kku naduvula spacing kodukkum.

3. Flexbox vs Grid

Feature	Flexbox	Grid
Layout type	One-dimensional	Two-dimensional
Main focus	Row or column	Rows and columns
Main class	`flex`	`grid`
Alignment	`justify-*`, `items-*`	`justify-*`, `items-*`, `place-*`
Common usage	Navbar, buttons, menus	Card layouts, galleries, dashboards
Responsive layout	`md:flex-row`	`md:grid-cols-3`

Easy-a remember pannunga:

- Navbar-la logo and menu items arrange panna → **flex**.
- Portfolio-la 6 skill cards arrange panna → **grid**.
- Card-kulla image and text align panna → **flex**.
- Dashboard-la multiple rows and columns create panna → **grid**.

4. Responsive Flexbox and Grid

Tailwind responsive prefixes use panni mobile, tablet, desktop layouts change pannalaam.

Responsive Flexbox

```
<div class="flex flex-col md:flex-row gap-4">
  <div class="bg-blue-500 p-4 text-white">Box 1</div>
  <div class="bg-green-500 p-4 text-white">Box 2</div>
</div>
```

- Mobile-la `flex-col` → Vertical layout.
- Medium screen and above-la `md:flex-row` → Horizontal layout.

Responsive Grid

```
<div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
  <div class="bg-blue-500 p-4 text-white">Card 1</div>
  <div class="bg-green-500 p-4 text-white">Card 2</div>
  <div class="bg-purple-500 p-4 text-white">Card 3</div>
</div>
```

- Mobile → 1 column.
- Small breakpoint and above → 2 columns.
- Large breakpoint and above → 3 columns.

PRACTICE TASK

- Create a navigation bar with a logo on the left and links on the right using Flexbox.

- Build a row of four cards using `flex`, `gap-*`, and `flex-wrap`.
- Create a responsive photo gallery using CSS Grid utilities.
- Center a login form horizontally and vertically.
- Build a dashboard with a sidebar and a main content area.

CHAPTER 05

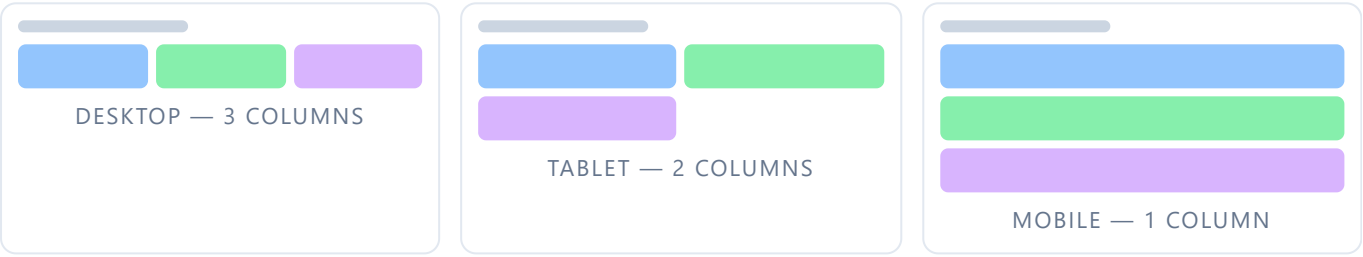
Responsive Design in Tailwind CSS

Mobile-first breakpoints — oru codebase, ellaa screen sizes-kum perfect layout.

1. Responsive Design na enna?

Responsive Design na, namma website mobile, tablet, laptop, desktop nu different screen sizes-la correct-ah display aagura maadhiri design panradhu.

Example: Desktop-la 3 cards side-by-side irukkalaam. Mobile-la adhe cards one-by-one vertical-ah display aagum.



- **Desktop:** Wide screen-la 3 columns or more display pannalaam.
- **Tablet:** Screen size-ku etha maadhiri 2 columns use pannalaam.
- **Mobile:** Content one column-la readable-ah display aagum.

2. Tailwind CSS Breakpoints

Tailwind-la responsive design create panna `sm:`, `md:`, `lg:`, `xl:`, `2xl:` prefixes use pannuvom.

Prefix	Default minimum width	Use
`sm:`	640px	Small screens and above
`md:`	768px	Tablets and above
`lg:`	1024px	Laptops and above
`xl:`	1280px	Large desktops and above
`2xl:`	1536px	Extra-large screens and above

IMPORTANT

Indha breakpoints **minimum widths**. Example, `md:` apply aana 768px and adhukku mela irukkura screen sizes-la apply aagum.

3. Responsive Prefix Eppadi Work Aagum?

EXAMPLE

```
<h1 class="text-base md:text-2xl lg:text-4xl">
  Welcome to My Website
```

```
</h1>
```

Explanation:

- `text-base` → Default-ah small text size.
- `md:text-2xl` → 768px and above-la larger text.
- `lg:text-4xl` → 1024px and above-la innum larger text.

Tailwind-la **mobile-first approach** use pannuvom. Prefix illaadha class default styles-ku apply aagum; responsive prefixes larger breakpoints-la styles-ai change pannum.

4. Responsive Width

`w-full`, `w-1/2`, `w-1/3` use panni screen size-ku etha maadhiri width change pannalaam.

```
<div class="w-full md:w-1/2 lg:w-1/3 bg-blue-500 p-4">
  Responsive Box
</div>
```

Screen	Width
Mobile	100%
Tablet (`md:`)	50%
Desktop (`lg:`)	33.33%

Mobile-la full width edukkum. Tablet-la half width; desktop-la one-third width edukkum.

5. Responsive Flexbox

Mobile-la vertical layout, desktop-la horizontal layout venumna:

```
<div class="flex flex-col md:flex-row gap-4">
  <div class="bg-blue-500 p-4 text-white">Box 1</div>
  <div class="bg-green-500 p-4 text-white">Box 2</div>
  <div class="bg-purple-500 p-4 text-white">Box 3</div>
</div>
```

- `flex` → Flexbox enable pannum.
- `flex-col` → Mobile-la vertical-ah arrange pannum.
- `md:flex-row` → 768px and above-la horizontal-ah arrange pannum.
- `gap-4` → Boxes-kku naduvula space kodukkum.

6. Responsive Grid

Cards, products, portfolio skills layout-ku Grid romba useful.

```
<div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
  <div class="bg-blue-500 p-4 text-white">Card 1</div>
  <div class="bg-green-500 p-4 text-white">Card 2</div>
  <div class="bg-purple-500 p-4 text-white">Card 3</div>
  <div class="bg-red-500 p-4 text-white">Card 4</div>
  <div class="bg-orange-500 p-4 text-white">Card 5</div>
  <div class="bg-pink-500 p-4 text-white">Card 6</div>
</div>
```

- `grid-cols-1` → Mobile-la 1 column.
- `sm:grid-cols-2` → 640px and above-la 2 columns.
- `lg:grid-cols-3` → 1024px and above-la 3 columns.
- `gap-4` → Cards-kku naduvula spacing.

7. Responsive Padding and Margin

Different screens-la spacing-ai adjust pannalaam.

```
<div class="p-4 md:p-8 lg:p-12">
  Responsive Spacing
</div>
```

- `p-4` → Default padding.
- `md:p-8` → Medium screens-la more padding.
- `lg:p-12` → Large screens-la innum more padding.

Innum sila examples:

Class	Meaning
`mt-4 md:mt-8`	Responsive top margin
`px-3 md:px-8`	Responsive horizontal padding
`mx-auto`	Horizontal margin auto
`max-w-7xl mx-auto`	Maximum width limit with centered content

8. Responsive Visibility

Certain elements-ai specific screen sizes-la hide or show pannalaam.

```
<!-- Mobile-la mattum visible -->
<div class="block md:hidden">
  Mobile Menu
</div>

<!-- Medium screens and above-la visible -->
<div class="hidden md:block">
  Desktop Menu
</div>
```

- `block md:hidden` → Mobile-la visible; 768px and above-la hidden.
- `hidden md:block` → Mobile-la hidden; 768px and above-la visible.

9. Responsive Image

Image parent container-ai exceed pannama display panna:

```
<img
  src="image.jpg"
  alt="Portfolio image"
  class="w-full h-auto object-cover"
/>
```

- `w-full` → Available width full-ah use pannum.
- `h-auto` → Image-oda aspect ratio preserve pannum.
- `object-cover` → Fixed-size container use pannumbodhu image-ai crop panni fill pannum.

Responsive page-ku HTML-la viewport meta tag-um important:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

10. Test Pannunga

TIP

Keela irukkura practice examples-ai browser-la different screen sizes-la (DevTools → Toggle Device Toolbar) test panni, layout eppadi change aagudhunu paarunga. Mobile-first classes eppadi override aagudhu-nu direct-ah observe panna mudiyum.

PRACTICE TASK

- Create a page that displays one card per row on mobile and three cards per row on desktop.
- Build a responsive navigation bar.
- Make a heading small on mobile and large on desktop.
- Create a responsive image gallery using `grid-cols-*` and breakpoint variants.
- Design a portfolio section that works on mobile, tablet, and desktop.

CHAPTER 06

Backgrounds and Visual Effects

Colors, gradients, images, borders, shadows, opacity, blur and glassmorphism.

Tailwind CSS-la Backgrounds and Visual Effects use panni website-ku background color, gradient, image, shadow, opacity, blur, border radius pondra visual styling effects apply pannalaam.

1. Background Color

Element-oda background color change panna `bg-*` classes use pannuvom.

Class	Explanation
`bg-red-500`	Red background
`bg-blue-500`	Blue background
`bg-green-500`	Green background
`bg-yellow-400`	Yellow background
`bg-white`	White background
`bg-black`	Black background
`bg-transparent`	Transparent background
`bg-gray-100`	Light gray background
`bg-gray-900`	Dark gray background

EXAMPLE

```
<div class="bg-blue-500 text-white p-6">
  Welcome to Tailwind CSS
</div>
```

- `bg-blue-500` → Background blue-a maarum.
- `text-white` → Text white-a irukkum.
- `p-6` → Content suththi padding varum.

2. Background Gradient

Rendu illa adhukku mela colors blend aagi display aaguradhukku gradient nu solluvom.

Class	Explanation
`bg-linear-to-r`	Left-to-right gradient
`bg-linear-to-l`	Right-to-left gradient
`bg-linear-to-t`	Bottom-to-top gradient
`bg-linear-to-b`	Top-to-bottom gradient
`from-blue-500`	Starting color
`via-purple-500`	Middle color
`to-pink-500`	Ending color

EXAMPLE

```
<div class="bg-linear-to-r from-blue-500 via-purple-500 to-pink-500 p-8 text-white">
  Gradient Background
</div>
```

> **NOTE**
> Indha gradient class names **Tailwind CSS v4 syntax**. Tailwind v3-la `bg-gradient-to-r` use pannuvanga.

3. Background Image

Element-ku background image set panna `bg-[url(...)]` syntax use pannalaam.

```
<div class="bg-[url('/images/bg.jpg')] bg-cover bg-center h-64">
</div>
```

Class	Explanation
`bg-cover`	Container full-ah cover pannum
`bg-contain`	Full image visible-a irukka fit pannum
`bg-center`	Image center-la position aagum
`bg-top`	Image top-la align aagum
`bg-no-repeat`	Image repeat aagadhu

Class	Explanation
`bg-repeat`	Image repeat aagum
`bg-fixed`	Background viewport-kku fixed-a irukkum

EXAMPLE

```
<section class="bg-[url('/images/hero.jpg')] bg-cover bg-center bg-no-repeat min-h-screen">
  <h1 class="text-white text-4xl font-bold p-8">
    My Portfolio
  </h1>
</section>
```

Explanation: Background image container-ai cover pannum; center-la position aagum, repeat aagadhu.

4. Background Size and Position

Class	Meaning
`bg-auto`	Natural background size
`bg-cover`	Container full-ah cover pannum
`bg-contain`	Image full-ah fit aagum
`bg-left`	Left position
`bg-right`	Right position
`bg-bottom`	Bottom position
`bg-no-repeat`	Repeat prevent pannum

5. Border Utilities

Element-oda border width and color control panna use pannuvom.

Class	Explanation
`border`	Default border width
`border-2`	2-unit border width
`border-4`	4-unit border width
`border-blue-500`	Blue border
`border-red-500`	Red border
`border-dashed`	Dashed border
`border-dotted`	Dotted border
`border-solid`	Solid border
`border-none`	Border remove pannum

EXAMPLE

```
<div class="border-2 border-blue-500 border-dashed p-6">
  Dashed Border Box
```

```
</div>
```

6. Border Radius

Corners-ai rounded shape-la maatha `rounded-*` classes use pannuvom.

Class	Explanation
`rounded-none`	Rounded corners illa
`rounded-sm`	Slight rounded
`rounded-md`	Medium rounded
`rounded-lg`	Large rounded
`rounded-xl`	Extra-large rounded
`rounded-2xl`	Innum periya rounding
`rounded-full`	Full rounded; circles/pills-ku useful

EXAMPLE

```
<div class="bg-green-500 text-white p-6 rounded-xl">
  Rounded Card
</div>
```

7. Box Shadow

Element-ku shadow effect kodukka `shadow-*` classes use pannuvom.

Class	Explanation
`shadow-2xs`	Very subtle shadow
`shadow-xs`	Extra-small shadow
`shadow-sm`	Small shadow
`shadow-md`	Medium shadow
`shadow-lg`	Large shadow
`shadow-xl`	Extra-large shadow
`shadow-2xl`	Very large shadow
`shadow-none`	Shadow remove pannum

EXAMPLE

```
<div class="bg-white p-6 rounded-xl shadow-lg">
  <h2 class="text-xl font-bold">My Card</h2>
  <p class="text-gray-600">This is a shadow card.</p>
</div>
```

Use case: Portfolio cards, product cards, login forms, dashboard panels.

8. Opacity

Element or background evvalavu transparent-a irukkanum-nu control pannalaam.

Class	Meaning
`opacity-0`	Completely transparent
`opacity-25`	25% opacity
`opacity-50`	50% opacity
`opacity-75`	75% opacity
`opacity-100`	Fully opaque
`bg-black/50`	Background color 50% opacity

EXAMPLE

```
<div class="bg-black/50 text-white p-6">
  Transparent Background
</div>
```

NOTE

`opacity-50` apply pannina element-oda content-um transparent aagum. Background mattum transparent-a venumna `bg-black/50` maadhiri **color opacity syntax** use pannunga.

9. Blur and Filter Effects

Images and backgrounds-ku visual effects create panna use aagum.

Class	Explanation
`blur-sm`	Small blur
`blur-md`	Medium blur
`blur-lg`	Large blur
`blur-none`	Blur remove pannum
`grayscale`	Black-and-white effect
`grayscale-0`	Normal colors
`brightness-50`	Brightness reduce pannum
`brightness-125`	Brightness increase pannum
`contrast-125`	Contrast increase pannum
`saturate-150`	Color saturation increase pannum

EXAMPLE

```
<img
  src="/images/photo.jpg"
  alt="Portfolio"
```

```
  class="w-64 rounded-xl blur-sm"
/>
```

Indha example image-ku small blur effect apply pannum.

10. Backdrop Effects

Element-kku pinnadi irukkura content-ai blur or adjust panna backdrop utilities use pannuvom. Glassmorphism designs-la idhu useful.

Class	Explanation
`backdrop-blur-sm`	Background content slight blur
`backdrop-blur-md`	Medium background blur
`backdrop-blur-lg`	Stronger background blur
`backdrop-brightness-75`	Behind content brightness reduce
`backdrop-saturate-150`	Behind content saturation increase

EXAMPLE

```
<div class="bg-white/20 backdrop-blur-lg border border-white/30 rounded-xl p-6">
  <h2 class="text-white text-2xl font-bold">
    Glass Effect Card
  </h2>
</div>
```

IMPORTANT

Backdrop blur clearly theriyanumna, element-kku pinnadi image or visible content irukkanum.

PRACTICE TASK

- Create a hero section using a gradient background.
- Add a background image with the correct size and position.
- Build a card with shadow, rounded corners, and a border.
- Create a transparent overlay on top of an image.
- Design a glassmorphism card using backdrop blur and transparency utilities.

CHAPTER 07

Typography and Text

Fonts, sizes, spacing, alignment, decoration, truncation and line clamping.

Tailwind CSS-la Typography and Text utilities use panni website-la text-oda font style, size, weight, spacing, alignment, decoration, overflow, responsiveness pondravatthai control pannalaam.

1. Font Family and Font Weight

Font family na text endha font style-la display aaganum-nu decide pannum. **Font weight** na text evvalavu thin-ah illa bold-ah irukkanum-nu control pannum.

Font Family Classes

Class	Explanation
`font-sans`	Sans-serif font
`font-serif`	Serif font
`font-mono`	Monospace font

Font Weight Classes

Class	Weight
`font-thin`	100
`font-light`	300
`font-normal`	400
`font-medium`	500
`font-semibold`	600
`font-bold`	700
`font-extrabold`	800
`font-black`	900

EXAMPLE

```
<h1 class="font-sans font-bold">
  Welcome to Tailwind CSS
</h1>

<p class="font-serif font-light">
  Learn web development.
</p>

<code class="font-mono">
  console.log("Hello");
</code>
```

`font-sans` font family-ai select pannum; `font-bold` text-ai bold-ah maathum.

2. Font Size and Line Height

`text-*` classes font size-ai control pannum. `leading-*` classes text lines-kku naduvula irukkura vertical spacing-ai control pannum.

Class	Explanation
`text-xs`	Extra-small text
`text-sm`	Small text
`text-base`	Normal text
`text-lg`	Large text

Class	Explanation
`text-xl`	Extra-large text
`text-2xl`	2X large text
`text-4xl`	Very large heading
`leading-none`	Line height 1
`leading-tight`	Compact line spacing
`leading-normal`	Normal line spacing
`leading-relaxed`	Relaxed line spacing
`leading-loose`	More line spacing

EXAMPLE

```
<h1 class="text-4xl font-bold leading-tight">
  Build Modern Websites
</h1>

<p class="text-base leading-relaxed">
  Tailwind CSS helps developers create
  attractive and responsive user interfaces.
</p>
```

- `text-4xl` → Heading size perusa irukkum.
- `leading-tight` → Heading lines close-ah irukkum.
- `leading-relaxed` → Paragraph padikka comfortable-ah spacing irukkum.

3. Letter Spacing and Text Alignment

Letter spacing na letters-kku naduvula irukkura space. **Text alignment** na text left, center, right-la eppadi display aaganum-nu decide pannum.

Letter Spacing

Class	Meaning
`tracking-tighter`	Romba close letters
`tracking-tight`	Close letters
`tracking-normal`	Normal spacing
`tracking-wide`	Extra letter spacing
`tracking-wider`	Innum adhigama spacing
`tracking-widest`	Maximum predefined spacing

Text Alignment

Class	Meaning
`text-left`	Left alignment

Class	Meaning
`text-center`	Center alignment
`text-right`	Right alignment
`text-justify`	Text-ai both sides align pannum
`text-start`	Text direction-oda starting side
`text-end`	Text direction-oda ending side

EXAMPLE

```
<h1 class="text-3xl font-bold tracking-wide text-center">
  MY PORTFOLIO
</h1>
```

Output: Large bold heading, wider letter spacing, center alignment.

4. Text Decoration and Text Transform

Text-ku underline, line-through, uppercase, lowercase pondra styles apply pannalaam.

Class	Explanation
`underline`	Text keezha line
`overline`	Text mela line
`line-through`	Text mela strike line
`no-underline`	Decoration remove pannum
`uppercase`	Ezhuthukkal ellam capital
`lowercase`	Ezhuthukkal ellam small
`capitalize`	Ovvoru word-oda first letter capital
`normal-case`	Original text casing preserve pannum

EXAMPLE

```
<a class="underline text-blue-600">
  Visit Website
</a>

<p class="line-through">Old Price: ₹1000</p>

<h2 class="uppercase">frontend developer</h2>
```

Result: Link underline-oda display aagum, old price strike aagum, heading FRONTEND DEVELOPER-nu display aagum.

5. Text Overflow and Truncation

Text container-oda width-ai thaandi pona, text-ai eppadi handle pannanum-nu indha utilities control pannum.

Class	Meaning
`truncate`	Overflow hide panni end-la ellipsis (...) kaattum
`text-ellipsis`	Ellipsis overflow style
`text-clip`	Text-ai clip pannum
`overflow-hidden`	Container-ai thaandi pogura content hide aagum
`whitespace-nowrap`	Text next line-ku wrap aagama irukkum

EXAMPLE

```
<p class="w-48 truncate">
  This is a very long paragraph that will be truncated.
</p>
```

Explanation: `w-48` width limit set pannum; `truncate` extra text-ai hide panni ellipsis kaattum.

> **NOTE**
> `text-ellipsis` mattum use pannina, ellaa cases-layum truncation automatic-ah nadakkaadhu. Width constraint and suitable overflow settings thevai.

6. Word Breaking and Whitespace

Long words, URLs, spaces, and line wrapping-ai control panna indha classes use aagum.

Word Breaking

Class	Meaning
`break-normal`	Normal word-breaking rules
`break-words`	Thevai pattal long words wrap aagum
`break-all`	Thevai pattal any character-kku appuram break aagum
`break-keep`	Certain languages-la words-ai unnecessary-ah break pannama preserve panna help pannum

Whitespace

Class	Meaning
`whitespace-normal`	Normal-ah wrap aagum
`whitespace-nowrap`	Wrap aagadhu
`whitespace-pre`	Spaces and line breaks preserve aagum
`whitespace-pre-line`	Line breaks preserve aagum; spaces collapse aagum
`whitespace-pre-wrap`	Spaces and line breaks preserve aagum; wrapping allow pannum

EXAMPLE

```
<p class="break-words">
  https://example.com/a-very-long-url-with-many-characters
</p>
```

```
<p class="whitespace-pre-line">
  First line
  Second line
  Third line
</p>
```

`break-words` long words container-ai thaandi pogama wrap panna help pannum. `whitespace-pre-line` source-la irukkura line breaks-ai preserve pannum.

7. Lists and List Markers

Lists-la bullet points, numbers, and marker position control panna use pannuvom.

Class	Meaning
`list-disc`	Bullet points
`list-decimal`	Numbered list
`list-none`	List markers remove pannum
`list-inside`	Marker content area-kulla irukkum
`list-outside`	Marker content area-vukku veliya irukkum

EXAMPLE

```
<ul class="list-disc list-inside">
  <li>HTML</li>
  <li>CSS</li>
  <li>Tailwind CSS</li>
</ul>

<ol class="list-decimal list-inside">
  <li>Learn basics</li>
  <li>Practice coding</li>
  <li>Build projects</li>
</ol>
```

Result: First list bullet points-oda display aagum; second list numbers-oda display aagum.

8. Custom Fonts

Google Fonts or local font files use panni custom typography set pannalaam. Font-ai project-la load pannitu, Tailwind-la font family utility use pannalaam.

EXAMPLE — GOOGLE FONTS IMPORT

```
<head>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?
family=Poppins:wght@400;600;700&display=swap"
        rel="stylesheet">
</head>
```

TAILWIND CSS V4-LA CUSTOM FONT THEME DEFINE PANNA

```
@import "tailwindcss";

@theme {
  --font-poppins: "Poppins", sans-serif;
}
```

HTML

```
<h1 class="font-poppins font-bold text-3xl">
  My Portfolio
</h1>
```

Explanation: Font load pannina piragu `font-poppins` utility use panni Poppins font apply pannalaam.

9. Responsive Typography

Different screen sizes-la text size and alignment-ai change panna responsive prefixes use pannuvom.

Class	Explanation
`text-base`	Default text size
`md:text-xl`	Medium breakpoint and above-la larger text
`lg:text-3xl`	Large breakpoint and above-la bigger text
`text-left`	Left alignment
`md:text-center`	Medium screens-la center alignment

EXAMPLE

```
<h1 class="text-2xl md:text-4xl lg:text-5xl font-bold">
  Frontend Developer
</h1>

<p class="text-sm md:text-base leading-relaxed">
  I build responsive websites using modern technologies.
</p>
```

- Mobile → `text-2xl` heading.
- Tablet (`md:`) → `text-4xl` .
- Desktop (`lg:`) → `text-5xl` .

Tailwind default breakpoints-la `md` 768px, `lg` 1024px. Custom configuration irundha values maaralaam.

10. Line Clamping

Line clamping na paragraph-ai specific number of lines-kulla limit panni, remaining text-ai hide pannuradhu.

Class	Meaning
`line-clamp-1`	One line
`line-clamp-2`	Two lines
`line-clamp-3`	Three lines

Class	Meaning
`line-clamp-4`	Four lines
`line-clamp-none`	Clamp remove pannum

EXAMPLE

```
<p class="line-clamp-3">
  Tailwind CSS is a utility-first CSS framework
  that helps developers create responsive user
  interfaces quickly. It offers reusable utility
  classes for spacing, colors, typography, and layouts.
</p>
```

Result: Paragraph maximum 3 lines-la display aagum; remaining text truncate aagum.

Idhu blog cards, product descriptions, and portfolio project summaries-la useful.

11. Quick Revision Cheat Sheet

Category	Important classes
Font family	`font-sans`, `font-serif`, `font-mono`
Font weight	`font-normal`, `font-semibold`, `font-bold`
Font size	`text-sm`, `text-base`, `text-xl`, `text-4xl`
Line height	`leading-none`, `leading-tight`, `leading-relaxed`
Letter spacing	`tracking-tight`, `tracking-wide`
Alignment	`text-left`, `text-center`, `text-right`
Decoration	`underline`, `line-through`, `no-underline`
Transform	`uppercase`, `lowercase`, `capitalize`
Overflow	`truncate`, `text-ellipsis`, `text-clip`
Word breaking	`break-normal`, `break-words`, `break-all`
Whitespace	`whitespace-normal`, `whitespace-nowrap`, `whitespace-pre-line`
Lists	`list-disc`, `list-decimal`, `list-none`
Responsive	`sm:text-lg`, `md:text-2xl`, `lg:text-4xl`
Line clamping	`line-clamp-1`, `line-clamp-3`, `line-clamp-none`

PRACTICE TASK

- Create a blog article layout with headings, paragraphs, and lists.
- Style a heading using custom font size, weight, and letter spacing.
- Make a paragraph readable using `leading-*` and `max-w-*`.
- Create a gradient-colored heading using background clipping and text transparency.
- Build a text truncation card using `truncate` and `line-clamp` utilities.

CHAPTER 08

States and Variants

hover, focus, group, peer, dark mode — condition-based styling-ku complete guide.

Tailwind CSS-la States and Variants na, user interaction, element position, form status, dark mode, screen settings pondra conditions-ku etha maadhiri CSS styles apply panradhu.

Example: Button mela mouse kondu pona blue background dark aaguradhu — idhu `hover:` variant use pannuradhunaala nadakkum.

1. hover:, focus:, active:

Indha variants user interaction-ku etha maadhiri styling apply pannum.

Variant	Explanation
`hover:`	Mouse element mela irukkumbodhu
`focus:`	Element keyboard focus or programmatic focus-la irukkumbodhu
`active:`	Element activate/click pannura nerathula

EXAMPLE

```
<button class="bg-blue-500 text-white px-4 py-2 rounded
               hover:bg-blue-700
               focus:ring-2 focus:ring-blue-300
               active:scale-95">
  Click Me
</button>
```

Explanation:

- `bg-blue-500` → Normal background blue.
- `hover:bg-blue-700` → Hover pannumbodhu dark blue.
- `focus:ring-2` → Focus-la ring display aagum.
- `active:scale-95` → Press pannumbodhu button konjam shrink aagum.

2. visited: and disabled:

visited:

Already visit pannina link-ku different style apply panna use pannuvom.

```
<a href="https://example.com"
   class="text-blue-600 visited:text-purple-600">
  Visit Website
</a>
```

- `text-blue-600` → Normal link blue.
- `visited:text-purple-600` → Visited link purple.

disabled:

Button or form control disabled state-la irukkumbodhu style apply pannum.

```
<button disabled
        class="bg-blue-500 text-white px-4 py-2
               disabled:bg-gray-400
               disabled:cursor-not-allowed">
  Submit
</button>
```

- `disabled:bg-gray-400` → Disabled button gray-a maarum.
- `disabled:cursor-not-allowed` → Interaction allowed illa-nu cursor indication kodukkum.

> **IMPORTANT**
> `disabled:` styling mattum button-ai disable pannaadhu. HTML-la `disabled` attribute-um irukkanum.

3. focus-within: and focus-visible:

focus-within:

Parent element-kulla irukkura edhaavadhu oru child focus aana, parent-ku style apply pannum.

```
<div class="border border-gray-300 rounded-lg p-3
            focus-within:border-blue-500
            focus-within:ring-2
            focus-within:ring-blue-200">
  <label for="username">Username</label>
  <input id="username" class="w-full outline-none"
         placeholder="Enter username" />
</div>
```

Explanation: Input-kulla click pannina, parent container border blue-a maarum.

focus-visible:

Keyboard navigation maadhiri focus indication thevaippadum situations-la styling apply pannum.

```
<button class="px-4 py-2 rounded
               focus-visible:outline-2
               focus-visible:outline-offset-2
               focus-visible:outline-blue-600">
  Keyboard Accessible Button
</button>
```

Difference:

- `focus:` → Element focus aana apply aagum.
- `focus-visible:` → Browser focus-visible rules-padi focus indicator kaattum; keyboard users-ku especially useful.

4. first:, last:, odd:, even:

Oru list-la element-oda position base panni styling apply pannalaam.

Variant	Meaning
`first:`	First child
`last:`	Last child

Variant	Meaning
`odd:`	Odd-numbered child
`even:`	Even-numbered child

EXAMPLE

```
<ul>
  <li class="first:font-bold first:text-blue-600">
    HTML
  </li>
  <li class="odd:bg-gray-100 even:bg-white p-2">
    CSS
  </li>
  <li class="odd:bg-gray-100 even:bg-white p-2">
    JavaScript
  </li>
  <li class="last:text-red-600 p-2">
    Tailwind CSS
  </li>
</ul>
```

- `first:font-bold` → First item bold.
- `last:text-red-600` → Last item red.
- `odd:bg-gray-100` → Odd-position items light gray.
- `even:bg-white` → Even-position items white.

Use case: Tables, product lists, menus, pricing plans.

5. group-hover: and group-focus:

Parent element-oda state base panni child element-ai style panna **group** use pannuvom.

EXAMPLE

```
<div class="group p-5 rounded-lg border hover:bg-blue-50">
  <h2 class="font-bold group-hover:text-blue-600">
    My Project
  </h2>

  <p class="text-gray-600 group-hover:text-blue-800">
    View my latest web development project.
  </p>
</div>
```

Eppadi work aagum?

- `group` → Parent-ai group container-ah mark pannum.
- `group-hover:text-blue-600` → Parent hover aana heading color change aagum.
- `group-hover:text-blue-800` → Parent hover aana paragraph color change aagum.

group-focus:

Parent group-kulla focus aagura element irukkumbodhu child style change panna use pannalaam.

```
<div class="group border p-4 rounded">
  <input class="peer border p-2"
         placeholder="Enter your name" />
  <p class="text-gray-500 group-focus-within:text-blue-600">
    Enter your details
  </p>
</div>
```

> **NOTE**
> `group-focus:` and `group-focus-within:` rendu same illa. Parent-oda own focus state venumna `group-focus:` ; group-kulla child focus aagumbodhu react panna `group-focus-within:` use pannunga.

6. peer-checked: and peer-invalid:

Oru form control-oda state base panni adhan sibling element-ku style apply panna **peer** use pannuvom.

peer-checked:

```
<label>
  <input type="checkbox" class="peer" />
  Accept terms and conditions

  <span class="text-gray-500 peer-checked:text-green-600">
    ✓ Accepted
  </span>
</label>
```

Checkbox checked aana, `peer-checked:text-green-600` sibling text-ai green-a maathum.

peer-invalid:

```
<input
  type="email"
  required
  placeholder="Enter email"
  class="peer border rounded p-2"
/>

<p class="hidden peer-invalid:block text-red-600">
  Please enter a valid email address.
</p>
```

- `peer` → Input state-ai sibling variants use panna allow pannum.
- `peer-invalid:block` → Input invalid-a irundha message display aagum.

> **IMPORTANT**
> `peer-*` variants normally subsequent sibling elements-ku work aagum; HTML order mukkiyam. Form validation state browser validation rules-padi determine aagum.

7. dark: — Dark Mode

Website dark theme-la irukkumbodhu different colors apply panna `dark:` use pannuvom.

```
<div class="bg-white text-gray-900
            dark:bg-gray-900 dark:text-white
            p-6 rounded-lg">
  <h2 class="text-2xl font-bold">
    Dark Mode Card
  </h2>

  <p class="text-gray-600 dark:text-gray-300">
    This card supports light and dark themes.
  </p>
</div>
```

- `bg-white` → Light background.
- `dark:bg-gray-900` → Dark mode background.
- `text-gray-900` → Light mode text.
- `dark:text-white` → Dark mode text.

> **NOTE**
> Tailwind CSS v4-la `dark:` default-ah system color scheme-ai follow pannum. Manual toggle venumna custom dark variant or theme class setup pannalaam. (Chapter 10-la detailed-ah paarpom.)

8. motion-safe: and motion-reduce:

User device-la reduced-motion preference enable pannirundha, animations-ai reduce panna use pannuvom.

Variant	Meaning
`motion-safe:`	Reduced-motion preference illaadha users-ku styles
`motion-reduce:`	Reduced-motion preference irukkura users-ku styles

EXAMPLE

```
<button class="bg-blue-500 text-white px-4 py-2 rounded
               motion-safe:transition
               motion-safe:hover:scale-105
               motion-reduce:transition-none">
  Get Started
</button>
```

- `motion-safe:transition` → Motion allow pannura users-ku transition.
- `motion-safe:hover:scale-105` → Hover animation.
- `motion-reduce:transition-none` → Reduced-motion preference irundha transition remove pannum.

Idhu accessibility-ku important.

9. Arbitrary Variants

Tailwind-la built-in variants mattum illaama, custom CSS selector condition-um use pannalaam. Idhuku **arbitrary variants** nu peyar.

SYNTAX

```
[&:nth-child(3)]:bg-red-500
```

EXAMPLE

```
<ul>
  <li class="[&:nth-child(3)]:bg-red-100 p-3">Item 1</li>
  <li class="[&:nth-child(3)]:bg-red-100 p-3">Item 2</li>
  <li class="[&:nth-child(3)]:bg-red-100 p-3">Item 3</li>
  <li class="[&:nth-child(3)]:bg-red-100 p-3">Item 4</li>
</ul>
```

Indha example-la ovvoru `li`-yum thanoda third-child condition match aana mattum red background apply aagum. Parent-kulla third item mattum style panna, selector-ai parent-level-la structure padi apply pannuradhu better.

More examples:

```
<div class="[&>p]:text-blue-600">
  <p>Blue paragraph</p>
  <p>Another blue paragraph</p>
</div>
```

`[&>p]:text-blue-600` → Andha element-oda direct child paragraphs-ku blue text apply pannum.

10. State-Based Styling for Forms and Components

Forms-la `required`, `invalid`, `valid`, `checked`, `disabled`, `placeholder-shown` pondra states-ai base panni styling apply pannalaam.

Common Form Variants

Variant	Meaning
`required:`	Required field
`optional:`	Optional field
`valid:`	Valid input
`invalid:`	Invalid input
`checked:`	Checked checkbox/radio
`placeholder-shown:`	Placeholder visible-a irukkumbodhu
`read-only:`	Read-only field
`disabled:`	Disabled control

EXAMPLE

```
<form class="max-w-sm space-y-4">
  <input
    type="email"
    required
    placeholder="Enter email"
    class="w-full border rounded-lg p-3
           valid:border-green-500
           invalid:border-red-500
           focus:ring-2 focus:ring-blue-300"
  />
```

```
  <button
    type="submit"
    class="bg-blue-600 text-white px-4 py-2 rounded-lg
           hover:bg-blue-700
           disabled:opacity-50"
  >
    Submit
  </button>
</form>
```

- `valid:border-green-500` → Input valid-a irundha green border.
- `invalid:border-red-500` → Input invalid-a irundha red border.
- `focus:ring-2` → Focus-la ring.
- `disabled:opacity-50` → Disabled button faded-a display aagum.

11. Quick Revision Cheat Sheet

Category	Classes
Mouse and click	`hover:`, `active:`
Focus	`focus:`, `focus-visible:`, `focus-within:`
Link/button state	`visited:`, `disabled:`
Child position	`first:`, `last:`, `odd:`, `even:`
Parent state	`group-hover:`, `group-focus:`
Sibling state	`peer-checked:`, `peer-invalid:`
Theme	`dark:`
Accessibility	`motion-safe:`, `motion-reduce:`
Custom selectors	`[&>p]:...`, `[&:nth-child(3)]:...`
Form state	`valid:`, `invalid:`, `required:`, `checked:`

PRACTICE TASK

- Create a button with different normal, hover, focus, and active states.
- Style a form input using `focus:` and `invalid:` variants.
- Create a card that changes its appearance when its parent is hovered using `group-hover:`.
- Build a list where the last item has a different border using `last:`.
- Create a disabled button using `disabled:` utilities.
- Style an element differently in dark mode using `dark:`.

CHAPTER 09

Transitions and Animations

Smooth motion, transforms, loaders, custom keyframes and reduced-motion support.

Tailwind CSS-la Transitions and Animations use pannina, website elements smooth-ah move aagum, rotate aagum, size change aagum, loading effects display aagum.

Example: Button hover pannumbothu color smooth-ah change aagurathu, card zoom aagurathu, spinner rotate aagurathu.

1. Transition Property and Duration

Transition na oru CSS property value change aagumbothu, instant-ah change aagama smooth-ah change aagurathu.

Tailwind class	Explanation
`transition`	Common properties smooth-ah change aagum
`transition-all`	Ella animatable properties-kum transition
`transition-colors`	Text, background, border colors smooth-ah change aagum
`transition-opacity`	Opacity smooth-ah change aagum
`transition-transform`	Transform changes smooth-ah nadakkum
`duration-150`	150 milliseconds
`duration-300`	300 milliseconds
`duration-500`	500 milliseconds
`duration-700`	700 milliseconds
`duration-1000`	1000 milliseconds (1 second)

EXAMPLE — BUTTON HOVER EFFECT

```
<button class="bg-blue-500 text-white px-6 py-3 rounded
               transition-colors duration-300
               hover:bg-green-500">
  Click Me
</button>
```

- `transition-colors` — color change smooth-ah nadakkum.
- `duration-300` — 300ms time edukkum.
- `hover:bg-green-500` — mouse hover pannumbothu green aagum.

2. Delay and Timing Functions

A. Transition Delay

Transition start aagurathukku munnaadi wait panna use pannuvom.

Class	Meaning
`delay-75`	75ms wait
`delay-150`	150ms wait
`delay-300`	300ms wait
`delay-500`	500ms wait
`delay-1000`	1 second wait

```
<button class="bg-purple-500 text-white px-5 py-3 rounded
               transition-colors duration-300 delay-150
               hover:bg-pink-500">
  Hover Me
</button>
```

B. Timing Functions

Transition eppadi speed increase/decrease aaganum-nu control pannum.

Class	Explanation
`ease-linear`	Constant speed
`ease-in`	Slow-ah start aagum
`ease-out`	Slow-ah finish aagum
`ease-in-out`	Start and end rendu pakkamum smooth
`ease`	Default easing curve

```
<div class="transition-transform duration-500 ease-in-out
            hover:scale-110">
  Hover this card
</div>
```

> **NOTE**
> `ease-*` timing function-a control pannum; `duration-*` animation/transition evvalavu neram nadakkanum-nu control pannum.

3. Transform — Translate, Rotate, Scale, Skew

Transform use panni element-oda position, angle, size, shape appearance maathalaam.

- **Translate (Move):** `translate-x-4`, `translate-y-4` — element-a horizontal or vertical direction-la move pannum.
- **Rotate (Turn):** `rotate-45`, `-rotate-12` — element-a clockwise or anticlockwise rotate pannum.
- **Scale (Resize):** `scale-110`, `scale-125`, `scale-95` — element-a perusa or sirusa maathum.
- **Skew (Tilt):** `skew-x-12`, `skew-y-6` — element-oda shape-a slant/tilt pannum.

Class	Result
`translate-x-4`	Right side move
`-translate-x-4`	Left side move
`translate-y-4`	Down move
`-translate-y-4`	Up move
`rotate-45`	45° clockwise rotation
`-rotate-45`	-45° rotation
`scale-110`	110% scale
`scale-50`	50% scale

Class	Result
`skew-x-12`	X-axis skew
`skew-y-6`	Y-axis skew

EXAMPLE — CARD HOVER ANIMATION

```
<div class="bg-white p-6 rounded-xl shadow-lg
            transition-transform duration-300
            hover:-translate-y-2 hover:scale-105">
  <h2 class="text-xl font-bold">Web Developer</h2>
  <p>Learn Tailwind CSS</p>
</div>
```

Hover pannumbothu card konjam mela move aagum and 5% perusa aagum.

4. Animation Utilities

Tailwind CSS-la predefined animation utilities irukku. Idhu continuous motion or attention effects create panna help pannum.

Class	Effect	Example use
`animate-spin`	Continuous rotation	Loading spinner
`animate-ping`	Expand aagi fade out	Notification indicator
`animate-pulse`	Fade in/out	Skeleton loading
`animate-bounce`	Up/down movement	Scroll indicator
`animate-none`	Animation stop	Disable animation

5. Built-in Animations

A. Spin

```
<div class="size-12 border-4 border-blue-500
            border-t-transparent rounded-full animate-spin">
</div>
```

Use: Loading spinner.

B. Ping

```
<div class="relative size-4">
  <span class="absolute size-4 rounded-full
               bg-green-400 animate-ping"></span>
  <span class="relative block size-4 rounded-full
               bg-green-600"></span>
</div>
```

Use: Live status or notification indicator.

C. Pulse

```
<div class="h-5 w-48 rounded bg-gray-300 animate-pulse"></div>
```

Use: Page load aagumbothu content placeholder display panna.

D. Bounce

```
<div class="text-4xl animate-bounce">⬇️</div>
```

Use: Scroll down indication or attention-grabbing icon.

6. Custom Keyframes

Built-in animations mattum illaama, namma own animation-um create pannalaam. Tailwind CSS v4-la CSS `@theme` use panni custom keyframes define pannalaam.

```
@import "tailwindcss";

@theme {
  --animate-float: float 2s ease-in-out infinite;

  @keyframes float {
    0%, 100% {
      transform: translateY(0);
    }

    50% {
      transform: translateY(-12px);
    }
  }
}
```

HTML

```
<div class="animate-float text-4xl">
  🚀
</div>
```

Explanation:

- `float` — animation name.
- `2s` — one cycle duration.
- `ease-in-out` — smooth start and finish.
- `infinite` — continuously repeat aagum.

> **NOTE**
> Tailwind CSS v3-la custom animations usually `tailwind.config.js` -oda `theme.extend.keyframes` and `theme.extend.animation` moolama configure pannuvom.

7. Hover Animation Effects

`hover:` variant-oda transition and transform combine pannina attractive UI create pannalaam.

```
<button class="bg-blue-600 text-white px-5 py-3
               rounded-lg transition-all duration-300
               hover:bg-purple-600 hover:scale-105
               hover:shadow-lg">
  Explore More
</button>
```

IMAGE ZOOM

```
<div class="overflow-hidden rounded-xl">
  <img src="image.jpg"
       class="w-full transition-transform
              duration-500 hover:scale-110"
       alt="Workspace" />
</div>
```

`overflow-hidden` use pannurathunaala zoom aagumbothu image container-ku veliya theriyadhu.

8. Loading Animations

API data load aagumbothu, image load aagumbothu, illa page content ready aagura varaikkum loading indicator show pannalaam.

Spinner Loading

```
<div class="flex items-center gap-3">
  <div class="size-8 rounded-full border-4
              border-gray-300 border-t-blue-600
              animate-spin">
  </div>
  <p>Loading ... </p>
</div>
```

Skeleton Loading

```
<div class="flex items-center gap-4 p-4">
  <div class="size-12 rounded-full bg-gray-300 animate-pulse"></div>

  <div class="flex-1 space-y-2">
    <div class="h-4 w-3/4 rounded bg-gray-300 animate-pulse"></div>
    <div class="h-4 w-1/2 rounded bg-gray-300 animate-pulse"></div>
  </div>
</div>
```

Difference:

- **Spinner** — loading process nadakkudhu-nu indicate pannum.
- **Skeleton** — content varapogura layout-a placeholder-ah kaattum.

9. Reduced-motion Accessibility

Sila users-ku excessive movement, spinning, bouncing effects uncomfortable-ah irukkalaam. System-la reduced-motion preference enable pannirundha, animations reduce or disable pannuradhu accessibility-ku important.

Tailwind-la `motion-reduce:` and `motion-safe:` variants use pannalaam.

DISABLE ANIMATION FOR REDUCED-MOTION USERS

```
<div class="animate-spin motion-reduce:animate-none">
  ⌛
</div>
```

Reduced motion preference enabled-na `animate-none` apply aagum.

APPLY ANIMATION ONLY WHEN MOTION IS ALLOWED

```
<button class="transition-transform duration-300
               motion-safe:hover:scale-110">
  Hover Me
</button>
```

`motion-safe:` use pannina user reduced motion request pannadha condition-la mattum hover scale effect apply aagum.

> **REMEMBER**
> Decorative animations reduce pannunga; important information animation mattum depend aaga koodadhu.

10. Quick Revision Cheat Sheet

Requirement	Tailwind class
Smooth color change	`transition-colors`
Smooth all properties	`transition-all`
Set duration	`duration-300`
Add delay	`delay-150`
Smooth timing	`ease-in-out`
Move element	`translate-x-4`
Rotate element	`rotate-45`
Increase size	`scale-110`
Tilt element	`skew-x-12`
Rotate continuously	`animate-spin`
Pulse effect	`animate-pulse`
Bounce effect	`animate-bounce`
Ping effect	`animate-ping`
Disable animation	`animate-none`
Respect reduced motion	`motion-reduce:animate-none`

> **PRACTICE TASK**
> - Create a button that smoothly changes color when hovered.
> - Build a card that moves upward and scales when hovered.
> - Create a spinning loading indicator using `animate-spin`.

- Build a notification indicator using `animate-ping`.
- Create a skeleton loading layout using `animate-pulse`.
- Define a custom floating animation using keyframes.
- Respect reduced-motion preferences using `motion-reduce:`.

CHAPTER 10

Advanced Tailwind CSS

Arbitrary values, theme tokens, custom utilities, components, plugins, performance and accessibility.

Advanced Tailwind CSS na, basic utility classes-a use pannuradha thaandi, custom designs, reusable components, design systems, performance optimization, accessibility maadhiri professional-level concepts learn pannuradhu.

Indha notes-la Tailwind CSS **v4 syntax** primary-ah use pannuvom. Thevaiyaana idathula Tailwind v3 difference-um mention pannirukken.

1. Arbitrary Values

Tailwind-la predefined values mattum use panna thevai illa. Namakku thevaiyaana custom value-a square brackets `[]` kulla kudukkalaam.

Syntax: `property-[value]`

Class	Meaning
`w-[350px]`	Width 350px
`h-[420px]`	Height 420px
`mt-[37px]`	Top margin 37px
`text-[22px]`	Font size 22px
`rounded-[18px]`	Border radius 18px
`bg-[#123456]`	Custom background color
`grid-cols-[200px_1fr]`	Custom grid columns

EXAMPLE

```
<div class="w-[350px] h-[200px] bg-[#123456]
            text-[22px] text-white rounded-[18px] p-5">
  Advanced Tailwind CSS
</div>
```

REMEMBER

Class name-kulla spaces irundha, many arbitrary-value syntaxes-la underscore `_` use pannalaam. Example: `grid-cols-[200px_1fr]`.

2. Arbitrary Properties

Tailwind-la direct utility class illadha CSS property-aiyum arbitrary property syntax moolama write pannalaam.

Syntax: `[property:value]`

```
<div class="[mask-type:luminance]
            [text-wrap:balance]
            [accent-color:#22c55e]">
  Custom CSS properties
</div>
```

Class	CSS equivalent
`[mask-type:luminance]`	`mask-type: luminance;`
`[text-wrap:balance]`	`text-wrap: balance;`
`[accent-color:#22c55e]`	`accent-color: #22c55e;`
`[writing-mode:vertical-rl]`	`writing-mode: vertical-rl;`

Use case: Rare CSS properties-ku custom stylesheet create pannama utility class use pannalaam. Aana repeated usage irundha custom utility create pannuradhu better.

3. Custom Theme Values

Tailwind v4-la `@theme` directive use panni custom colors, fonts, spacing, breakpoints and animation tokens define pannalaam.

```
@import "tailwindcss";

@theme {
  --color-brand: #16a34a;
  --font-display: "Poppins", sans-serif;
  --breakpoint-3xl: 120rem;
  --radius-card: 1.25rem;
}
```

HTML

```
<div class="bg-brand font-display rounded-card">
  My Custom Theme
</div>
```

Inga:

- `--color-brand` → `bg-brand`, `text-brand` maadhiri utilities.
- `--font-display` → `font-display`.
- `--breakpoint-3xl` → `3xl:` responsive variant.
- `--radius-card` → `rounded-card`.

> **NOTE**
> Tailwind v3-la indha maadhiri values-a `tailwind.config.js`-la `theme.extend` use panni configure pannuvom.

4. CSS Variables with Utility Classes

CSS variables na oru value-a variable-la store panni, different places-la reuse pannuradhu.

```
<div style="--card-color: #0f172a;"
     class="bg-[var(--card-color)] text-white p-6 rounded-xl">
  CSS Variables
</div>
```

USING VARIABLES FOR REUSABLE COLORS

```
:root {
  --primary: #2563eb;
  --surface: #f8fafc;
}
```

```
<button class="bg-[var(--primary)] text-white px-5 py-2 rounded-lg">
  Submit
</button>

<div class="bg-[var(--surface)] p-5">
  Content
</div>
```

Syntax	Purpose
`bg-[var(--primary)]`	Variable background color
`text-[var(--primary)]`	Variable text color
`w-[var(--panel-width)]`	Variable width
`rounded-[var(--card-radius)]`	Variable border radius

> **TIP**
> Theme-wide design tokens-ku `@theme` use pannunga. Local component-specific values-ku ordinary CSS variables useful-ah irukkum.

5. Custom Utilities and Variants

A. Custom Utility

Repeated-ah same CSS styles use pannumbodhu, Tailwind v4-la `@utility` define pannalaam.

```
@import "tailwindcss";

@utility content-auto {
  content-visibility: auto;
}
```

HTML

```
<section class="content-auto">
  Page content
</section>
```

Innum oru example:

```
@utility text-shadow-soft {
  text-shadow: 0 2px 8px rgb(0 0 0 / 20%);
}
```

```
<h1 class="text-4xl font-bold text-shadow-soft">
  Welcome
</h1>
```

B. Custom Variant

Variant na `hover:`, `focus:` maadhiri oru condition-based style modifier.

```
@import "tailwindcss";

@custom-variant theme-dark (&:where([data-theme="dark"] *));
```

HTML

```
<div data-theme="dark">
  <p class="text-black theme-dark:text-white">
    Theme-based text
  </p>
</div>
```

Custom variants define pannumbodhu selector correct-ah apply aagudha-nu test pannunga.

6. Reusable Components

Oru design-a multiple places-la use pannumbodhu same classes-a repeated-ah ezhuthuradhukku badhila component create pannalaam.

EXAMPLE — REACT COMPONENT

```
function PrimaryButton({ children }) {
  return (
    <button
      className="rounded-lg bg-blue-600 px-5 py-3
                 font-semibold text-white
                 transition-colors duration-300
                 hover:bg-blue-700
                 focus-visible:outline-2
                 focus-visible:outline-offset-2
                 focus-visible:outline-blue-600"
    >
      {children}
    </button>
  );
}

export default PrimaryButton;
```

USE

```
<PrimaryButton>Login</PrimaryButton>
```

```
<PrimaryButton>Register</PrimaryButton>
```

Advantages:

- Duplicate code reduce aagum.
- Design consistent-ah irukkum.
- Future-la color or spacing change panna easy.
- Large projects maintain pannuradhu simple aagum.

Plain HTML project-na, reusable sections-ku template system, server-side partials, or JavaScript components use pannalaam.

7. Dark-Mode Strategies

Website light theme and dark theme support panna dark mode use pannuvom. Tailwind-la common-ah rendu strategies irukku.

A. System Preference

User device dark mode enable pannirundha adhai respect pannum.

```
<div class="bg-white text-gray-900
            dark:bg-gray-900 dark:text-white p-6">
  Dark Mode Support
</div>
```

B. Manual Theme Toggle

User button click panni light/dark theme choose pannalaam.

TAILWIND V4 CSS

```
@import "tailwindcss";

@custom-variant dark (&:where(.dark, .dark *));
```

HTML STRUCTURE

```
<div class="dark">
  <main class="min-h-screen bg-white p-6
               text-black dark:bg-gray-950
               dark:text-white">
    Dark theme content
  </main>
</div>
```

Indha example-la `.dark` class manually add pannirukkom. Real project-la JavaScript or framework state use panni toggle pannalaam.

Strategy	Best use
System preference	Device theme automatic-ah follow panna
Manual class	User-ku theme toggle option kudukka
Theme attribute	Multiple themes maintain panna

8. Plugin Development

Plugins moolama Tailwind-ku additional utilities, components or variants add pannalaam. Custom utilities mattum thevaipatta v4-la `@utility` pothumaanadhu; complex extensions-ku plugin approach useful.

TAILWIND CSS V3 PLUGIN EXAMPLE

```
// tailwind.config.js
const plugin = require("tailwindcss/plugin");

module.exports = {
  content: ["./src/**/*.{html,js,jsx}"],
  plugins: [
    plugin(function ({ addUtilities }) {
      addUtilities({
        ".text-shadow": {
          textShadow: "2px 2px 4px #00000040",
        },
      });
    }),
  ],
};
```

USE

```
<h1 class="text-shadow text-3xl font-bold">
  Custom Plugin
</h1>
```

> **IMPORTANT**
> Idhu Tailwind v3 configuration example. Tailwind v4-la normal custom utility-ku `@utility` -a prefer pannalaam; v3 configuration-ai v4 project-la direct-ah copy panna vendam.

9. Content Detection and Production Builds

Tailwind unga HTML, JavaScript and component files-la use pannirukkura class names-a scan panni, thevaiyaana CSS-a generate pannum.

```
<h1 class="text-3xl font-bold text-blue-600">
  Hello Tailwind
</h1>
```

Indha classes detect aagi output CSS-la include aagum.

Common Issue: Dynamic Class Names

```
// Risky: full class names source-la irukkaama pogalaam
<div className={`bg-${color}-500`}>
  Content
</div>
```

Tailwind class names-a source files-la detect pannuradhunaala, dynamic-ah construct pannina full class name detect aagama pogalaam.

BETTER APPROACH

```
const colors = {
  blue: "bg-blue-500",
  green: "bg-green-500",
  red: "bg-red-500",
};

<div className={colors[color] ?? colors.blue}>
  Content
</div>
```

Inga complete class names source-la irukkuradhu nala detection reliable-ah irukkum.

Production Build

- Development mode-la styles and layout test pannunga.
- Production build generate pannunga.
- Build output-la required CSS include aagirukkaa verify pannunga.
- Unused CSS-ai manually delete panna vendam; Tailwind production workflow generated output-ai optimize pannum.

Tailwind v4-la `@import "tailwindcss";` use pannuradhu normal setup. Legacy v3-la content paths and `tailwind.config.js` mukkiyam.

10. Performance Optimization

Fast website-ku CSS mattum illaama images, JavaScript and page rendering-um optimize pannanum.

Technique	Benefit
Static class names use pannunga	CSS detection reliable
Reusable components	Duplicate markup reduce
Unnecessary animations avoid pannunga	Rendering smooth-ah irukkum
Images compress pannunga	Page loading fast
Correct responsive utilities	Different screen sizes-ku better layout
Production build test pannunga	Deployment issues identify panna
Large dependencies avoid pannunga	Bundle size reduce aagum

EXAMPLE — RESPONSIVE IMAGE

```
<img
  src="/images/portfolio.webp"
  alt="Portfolio preview"
  class="h-auto w-full rounded-xl object-cover"
  loading="lazy"
/>
```

`loading="lazy"` below-the-fold images loading-a defer panna help pannum. First visible hero image-ku lazy loading use panna vendiya avasiyam illai; adhu initial display-a delay pannalaam.

11. Design Tokens

Design tokens na project full-ah consistent-ah use panna vendiya colors, spacing, typography, border radius and shadows maadhiri shared values.

```
@import "tailwindcss";

@theme {
  --color-primary: #2563eb;
  --color-surface: #f8fafc;
  --color-heading: #0f172a;

  --font-display: "Poppins", sans-serif;

  --radius-card: 1rem;
}
```

USE

```
<section class="bg-surface p-6 rounded-card">
  <h2 class="font-display text-heading text-2xl">
    My Portfolio
  </h2>

  <button class="bg-primary rounded-lg px-4 py-2 text-white">
    View Projects
  </button>
</section>
```

Why design tokens?

- Entire website-ku same color system maintain pannalaam.
- Theme update easy.
- Components consistent-ah irukkum.
- Team projects-la design changes manage pannuradhu simple.

12. Accessibility and Keyboard Focus

Website mouse use pannuravangalukku mattum illaama, keyboard and assistive technologies use pannuravangalukkum accessible-ah irukkanum.

A. Keyboard Focus

```
<button
  class="rounded-lg bg-blue-600 px-5 py-3 text-white
         focus-visible:outline-2
         focus-visible:outline-offset-4
         focus-visible:outline-blue-600">
  Submit
</button>
```

`focus-visible:` keyboard focus irukkumbodhu visible focus indicator show panna useful.

B. Form Labels

```
<label for="email" class="mb-2 block font-medium">
  Email address
</label>

<input
  id="email"
  type="email"
  class="w-full rounded-lg border border-gray-300
         p-3 focus-visible:outline-2
         focus-visible:outline-blue-600"
/>
```

Accessibility Checklist

- Buttons and links keyboard-la operate aaganum.
- Focus indicator visible-ah irukkanum.
- Text/background contrast sufficient-ah irukkanum.
- Images-ku meaningful alt text kudukkanum.
- Form inputs-ku labels irukkanum.
- Color mattum use panni error/success convey panna vendam.
- Decorative animations-ku reduced-motion support consider pannunga.

13. Debugging Common Styling Issues

Tailwind-la class apply aagala-na, indha checklist follow pannunga.

1. **Class spelling check:** `bg-blu-500` thappu; `bg-blue-500` correct.
2. **Source detection check:** Dynamic class names use pannirundha, full class names source-la irukkaa verify pannunga.
3. **CSS import check:** Tailwind v4 project-la `@import "tailwindcss";` irukkaa check pannunga.
4. **Responsive variants check:** `md:flex`, `lg:grid-cols-3` maadhiri classes breakpoint-ku match aagudha verify pannunga.
5. **Hover and focus check:** `hover:` mouse hover-la mattum activate aagum; `focus:` focus state-la activate aagum.
6. **CSS conflict check:** Browser DevTools → Inspect → Computed Styles-la endha CSS property override aagudhu-nu paarunga.

14. Quick Revision Cheat Sheet

Topic	Syntax / example
Arbitrary value	`w-[350px]`
Arbitrary property	`[mask-type:luminance]`
Custom theme	`@theme { --color-brand: #16a34a; }`
CSS variable	`bg-[var(--primary)]`
Custom utility	`@utility content-auto { ... }`
Custom variant	`@custom-variant ...`
Dark mode	`dark:bg-gray-950`

Topic	Syntax / example
Custom component	React component with `className`
Dynamic class fix	Map complete class names
Design token	`--color-primary`
Keyboard focus	`focus-visible:outline-2`
Reduced motion	`motion-reduce:animate-none`

PRACTICE TASK

- Create a custom-width card using `w-[350px]`.
- Apply an arbitrary CSS property using `[mask-type:luminance]`.
- Define custom brand colors and fonts using `@theme`.
- Use CSS variables for reusable colors and dimensions.
- Create a custom utility using `@utility`.
- Build reusable React components with Tailwind classes.
- Implement a light and dark theme switch.
- Fix dynamically generated class names so Tailwind detects them.
- Create a design-token system for a portfolio website.
- Add keyboard focus indicators and test the production build.

CHAPTER 11

Forms and Interactivity

Inputs, validation, cursors, scrolling, sticky headers, dropdowns and modals.

Forms and Interactivity na website-la users input enter pannuradhu, options select pannuradhu, checkbox click pannuradhu, dropdown open pannuradhu, modal display pannuradhu, scroll pannuradhu pondra interactions-a style pannuradhu.

Indha notes-la Tailwind CSS **v4 syntax** use pannuvom. Examples beginner-friendly-ah irukkum.

1. Input, Textarea, Select and Checkbox Styling

A. Input Field

```
<input
  type="text"
  placeholder="Enter your name"
  class="w-full rounded-lg border border-gray-300
         px-4 py-3 outline-none
         focus:border-blue-500
         focus:ring-2 focus:ring-blue-200"
/>
```

- `w-full` — parent width full-ah edukkum.
- `border-gray-300` — default border color.
- `focus:border-blue-500` — input focus aagumbothu blue border.
- `focus:ring-2` — focus ring display aagum.

B. Textarea

```
<textarea
  rows="4"
  placeholder="Write your message ... "
  class="w-full resize-y rounded-lg border
         border-gray-300 p-3
         focus:border-blue-500
         focus:ring-2 focus:ring-blue-200"
></textarea>
```

`resize-y` use pannina textarea height mattum resize panna mudiyum.

C. Select Dropdown

```
<select
  class="w-full rounded-lg border border-gray-300
         bg-white px-4 py-3
         focus:outline-2 focus:outline-blue-500"
>
  <option>Select a course</option>
  <option>HTML</option>
  <option>CSS</option>
  <option>JavaScript</option>
</select>
```

D. Checkbox

```
<label class="flex items-center gap-2">
  <input type="checkbox" class="size-5 accent-blue-600" />
  <span>I agree to the terms</span>
</label>
```

Class	Use
`border-gray-300`	Border color
`rounded-lg`	Rounded corners
`p-3`	Inner spacing
`focus:ring-2`	Focus ring
`resize-y`	Vertical resizing
`accent-blue-600`	Checkbox accent color

2. Placeholder and File-Input Styling

A. Placeholder Styling

`placeholder:` variant use panni placeholder text-oda color and style change pannalaam.

```
<input
  type="text"
  placeholder="Enter username"
  class="rounded-lg border p-3
```

```
        placeholder:text-gray-400
        placeholder:italic
        focus:outline-blue-500"
/>
```

Useful classes:

- `placeholder:text-gray-400`
- `placeholder:italic`
- `placeholder:opacity-60`

B. File Input Styling

`file:` variant use panni file upload button-a customize pannalaam.

```
<input
  type="file"
  accept="image/*"
  class="w-full rounded-lg border border-gray-300
         p-2 text-sm
         file:mr-4 file:rounded-md
         file:border-0 file:bg-blue-600
         file:px-4 file:py-2
         file:font-semibold file:text-white
         hover:file:bg-blue-700"
/>
```

Result: Default file upload button-ku badhila blue custom button maadhiri display aagum.

3. Form Validation States

Form validation na user correct data enter pannirukkaangala-nu check pannuradhu. Tailwind-la `required:`, `invalid:`, `valid:`, `disabled:` maadhiri state variants irukku. Indha classes **styling mattum pannum**; actual validation browser HTML validation or JavaScript moolama nadakkum.

```
<form>
  <label for="email" class="mb-2 block font-medium">
    Email
  </label>

  <input
    id="email"
    type="email"
    required
    placeholder="you@example.com"
    class="w-full rounded-lg border border-gray-300 p-3
           focus:outline-2 focus:outline-blue-500
           invalid:border-red-500
           valid:border-green-500"
  />

  <button
    type="submit"
    class="mt-4 rounded-lg bg-blue-600 px-5 py-2 text-white
           hover:bg-blue-700
           disabled:cursor-not-allowed disabled:opacity-50"
```

```
  >
    Submit
  </button>
</form>
```

Variant	Meaning
`required:`	Required field state
`invalid:`	Invalid input state
`valid:`	Valid input state
`disabled:`	Disabled control state
`focus:`	Element focus state
`placeholder:`	Placeholder styling

> **NOTE**
> `type="email"` -ku browser validation irukkum. Email field empty-ah irundhaal, `required` attribute irukkumbodhu submit panna mudiyadhu.

4. accent-* and caret-*

A. Accent Color

Checkbox, radio button, range slider pondra controls-oda accent color change pannum.

```
<input type="checkbox" checked class="size-5 accent-green-600" />

<input type="radio" name="course" class="accent-blue-600" />

<input type="range" class="w-48 accent-purple-600" />
```

B. Caret Color

Input-la typing pannumbodhu blink aagura cursor color-a control pannum.

```
<input
  type="text"
  placeholder="Start typing ... "
  class="rounded border p-3 caret-red-500
         focus:outline-blue-500"
/>
```

Class	Effect
`accent-blue-600`	Form control accent blue
`accent-green-600`	Form control accent green
`caret-red-500`	Typing cursor red
`caret-transparent`	Cursor visually transparent

5. Cursor and Pointer Events

A. Cursor Utilities

Class	Meaning
`cursor-pointer`	Clickable pointer cursor
`cursor-not-allowed`	Action allowed illa
`cursor-wait`	Wait cursor
`cursor-text`	Text selection cursor
`cursor-grab`	Grab interaction
`cursor-grabbing`	Dragging interaction

```
<button class="cursor-pointer rounded bg-blue-600 px-4 py-2 text-white">
  Click Me
</button>

<button disabled class="cursor-not-allowed opacity-50">
  Disabled
</button>
```

B. Pointer Events

`pointer-events-*` element mouse/touch pointer events-a receive pannuma-nu control pannum.

```
<div class="pointer-events-none opacity-50">
  This overlay ignores pointer events
</div>
<div class="pointer-events-auto">
  Pointer interactions are enabled
</div>
```

> **IMPORTANT**
> `pointer-events-none` use pannina element-ku pointer click pass aagum; aana keyboard focus or accessibility automatically disable aagathu. Truly disabled button-ku `disabled` attribute use pannunga.

6. Selection and Touch-Action Utilities

A. Text Selection

Class	Use
`select-none`	Text select panna mudiyadhu
`select-text`	Text select pannalaam
`select-all`	Text-a select pannumbodhu full content select
`select-auto`	Browser default selection behavior

```
<p class="select-all">
  Click and select this complete sentence.
```

```
</p>
```

`select-none` decorative controls-la useful. Aana normal text and useful content selection-a unnecessary-ah block panna vendam.

B. Touch Action

Mobile-la swipe, pan and zoom behavior-a control panna touch-action utilities use pannalaam.

```
<div class="touch-pan-y overflow-auto">
  Vertical scrolling content
</div>
```

Class	Meaning
`touch-auto`	Browser default touch behavior
`touch-none`	Browser touch gestures disable
`touch-pan-x`	Horizontal pan
`touch-pan-y`	Vertical pan
`touch-manipulation`	Common touch gestures; double-tap zoom behavior restrictions

> **IMPORTANT**
> `touch-none` -a carefully use pannunga; mobile users-ku scrolling and zooming kashtamaagalaam.

7. Scroll Behavior and Scroll Snapping

A. Smooth Scrolling

```
<html class="scroll-smooth">
  <body>
    <a href="#projects">Go to Projects</a>

    <section class="min-h-screen">Home</section>

    <section id="projects" class="min-h-screen">
      Projects
    </section>
  </body>
</html>
```

Anchor link click pannumbodhu smooth scrolling nadakkum. User reduced-motion preference irundha smooth scrolling-a disable pannuradhum consider pannunga.

B. Scroll Snapping

Horizontal gallery-la each card neat-ah align aaga use pannalaam.

```
<div class="flex snap-x snap-mandatory gap-4
            overflow-x-auto scroll-smooth p-4">
  <div class="w-64 shrink-0 snap-start rounded-xl bg-blue-200 p-8">
    Card 1
  </div>
```

```
  <div class="w-64 shrink-0 snap-start rounded-xl bg-green-200 p-8">
    Card 2
  </div>

  <div class="w-64 shrink-0 snap-start rounded-xl bg-purple-200 p-8">
    Card 3
  </div>
</div>
```

Class	Meaning
`overflow-x-auto`	Horizontal scroll
`snap-x`	Horizontal snapping
`snap-mandatory`	Scroll stop points enforce
`snap-start`	Card start position-la snap
`snap-center`	Card center-la snap
`scroll-smooth`	Smooth scroll behavior

8. Sticky Headers and Dropdown Menus

A. Sticky Header

`sticky` element scroll pannumbodhu specified position-la stick aagum.

```
<header class="sticky top-0 z-50 bg-white shadow-md">
  <nav class="mx-auto flex max-w-6xl items-center justify-between p-4">
    <h1 class="text-xl font-bold">My Portfolio</h1>

    <div class="flex gap-4">
      <a href="#home">Home</a>
      <a href="#about">About</a>
      <a href="#contact">Contact</a>
    </div>
  </nav>
</header>
```

- `sticky` — scroll time-la stick aagum.
- `top-0` — viewport top-la position.
- `z-50` — other content mela display aaga help pannum.
- `shadow-md` — header-ku shadow.

DEBUG TIP

Sticky work aagala-na parent element-oda overflow settings and available scroll area-a check pannunga.

B. Dropdown Menu

Pure HTML/CSS dropdown-ku `group-hover:` use pannalaam. Keyboard interaction-kum support venumna `<details>` approach simple-ah irukkum.

```
<details class="group relative inline-block">
  <summary class="cursor-pointer list-none rounded-lg
```

```
                bg-blue-600 px-4 py-2 text-white">
    Services ▾
  </summary>

  <div class="absolute left-0 top-full z-10 mt-2
              w-48 rounded-lg border bg-white p-2 shadow-lg">
    <a href="#web" class="block rounded p-2 hover:bg-gray-100">
      Web Development
    </a>
    <a href="#design" class="block rounded p-2 hover:bg-gray-100">
      UI Design
    </a>
  </div>
</details>
```

`<summary>` click pannumbodhu dropdown open/close aagum. Idhu JavaScript illama work aagum.

9. Modal and Dialog Layouts

Modal na current page mela overlay-la open aagura dialog. Example: Login form, confirmation box, settings panel.

FUNCTIONAL NATIVE DIALOG

```
<button
  onclick="document.getElementById('myDialog').showModal()"
  class="rounded-lg bg-blue-600 px-5 py-3 text-white">
  Open Modal
</button>

<dialog
  id="myDialog"
  class="m-auto w-[90%] max-w-md rounded-xl
         p-6 shadow-xl backdrop:bg-black/50">
  <h2 class="text-xl font-bold">Welcome!</h2>

  <p class="my-4 text-gray-600">
    This is a modal dialog.
  </p>

  <form method="dialog">
    <button class="rounded-lg bg-gray-900 px-4 py-2 text-white">
      Close
    </button>
  </form>
</dialog>
```

Inga Tailwind layout styles kudukkum; native HTML `<dialog>` and small JavaScript inline handler modal open pannum.

Accessibility tips:

- Dialog-ku meaningful title irukkanum.
- Keyboard-la close and operate panna mudiyanum.
- Focus dialog-kulla correct-ah manage aaganum.
- Modal open aagumbodhu background content interact panna mudiyadha behavior provide pannunga.

- Real project-la custom modal build pannumbodhu focus return and Escape key behavior test pannunga. Native `<dialog>`-oda `showModal()` indha behavior-la pala vishayangalai browser moolama handle pannum.

10. Quick Revision Cheat Sheet

Requirement	Tailwind class / HTML
Input focus border	`focus:border-blue-500`
Placeholder color	`placeholder:text-gray-400`
File button styling	`file:bg-blue-600`
Invalid field	`invalid:border-red-500`
Checkbox color	`accent-green-600`
Typing cursor	`caret-red-500`
Clickable cursor	`cursor-pointer`
Ignore pointer events	`pointer-events-none`
Disable text selection	`select-none`
Vertical touch pan	`touch-pan-y`
Smooth scroll	`scroll-smooth`
Horizontal snap	`snap-x snap-mandatory`
Sticky header	`sticky top-0 z-50`
Dropdown	`<details>` + `<summary>`
Modal	`<dialog>` + `showModal()`

PRACTICE TASK

- Create a custom-width card using `w-[350px]`.
- Apply an arbitrary CSS property using `[mask-type:luminance]`.
- Define custom brand colors and fonts using `@theme`.
- Use CSS variables for reusable colors and dimensions.
- Create a custom utility using `@utility`.
- Build reusable React components with Tailwind classes.
- Implement a light and dark theme switch.
- Fix dynamically generated class names so Tailwind detects them.
- Create a design-token system for a portfolio website.
- Add keyboard focus indicators and test the production build.

CHAPTER 12

50 Interview Questions and Answers

All 10 topics covered — beginner fundamentals to advanced concepts, forms, animations, accessibility and production optimization.

Section 1: Introduction to Tailwind CSS

1. What is Tailwind CSS?

Answer: Tailwind CSS is a utility-first CSS framework that allows developers to style HTML elements directly using predefined utility classes.

Example:

```
<h1 class="text-3xl font-bold text-blue-600">
  Hello World
</h1>
```

2. What is utility-first CSS?

Answer: Utility-first CSS means using small, single-purpose classes to style individual properties instead of writing custom CSS for every component.

Examples: `p-4`, `text-center`, `bg-blue-500`, and `rounded-lg`.

3. What are the advantages of Tailwind CSS?

Answer:

- Faster UI development.
- Consistent spacing and colors.
- Responsive design utilities.
- Easy hover and focus styling.
- Reduced need for custom CSS.
- Reusable components when combined with frameworks such as React.

4. How is Tailwind CSS different from Bootstrap?

Answer: Bootstrap provides predefined components such as navbars and buttons, while Tailwind provides low-level utility classes to create custom designs.

Tailwind CSS	Bootstrap
Utility-first framework	Component-oriented framework
Highly customizable layouts	Ready-made component styles
Classes are composed in markup	Components often use predefined styles

5. How do you install Tailwind CSS?

Answer: In a Tailwind CSS v4 project, install the required packages and configure the build tool or CLI. For example, using the Vite integration:

```
npm install tailwindcss @tailwindcss/vite
```

Then add the Tailwind Vite plugin and import Tailwind in your CSS:

```
@import "tailwindcss";
```

The exact setup depends on whether the project uses Vite, the CLI, PostCSS, or another supported integration.

Section 2: Core Utility Classes

6. What are utility classes in Tailwind CSS?

Answer: Utility classes apply individual styles such as spacing, colors, borders, sizing, and typography.

```
<div class="m-4 p-6 bg-gray-100 rounded-xl">
  Content
</div>
```

7. What is the difference between margin and padding?

Answer: Margin creates space outside an element's border, while padding creates space inside the border, between the content and the border.

- `m-4` — margin on all sides.
- `mt-4` — top margin.
- `p-4` — padding on all sides.
- `px-4` — horizontal padding.
- `py-4` — vertical padding.

8. How do you apply background and text colors?

Answer: Use `bg-*` for backgrounds and `text-*` for text colors.

```
<div class="bg-blue-600 p-4 text-white">
  Welcome
</div>
```

9. How do you add borders, rounded corners, and shadows?

Answer: Use `border`, `rounded-*`, and `shadow-*` utilities.

```
<div class="rounded-xl border border-gray-300 p-5 shadow-lg">
  Profile Card
</div>
```

10. How do you control element width and height?

Answer: Use sizing utilities such as `w-*`, `h-*`, `min-w-*`, and `max-w-*`.

```
<div class="h-40 w-64 max-w-full bg-green-500"></div>
```

Arbitrary values are also supported, for example `w-[350px]`.

Section 3: Flexbox and Grid

11. How do you create a Flexbox layout?

Answer: Use `flex` to enable Flexbox and utilities such as `justify-between`, `items-center`, and `gap-4` to arrange the elements.

```
<nav class="flex items-center justify-between gap-4">
  <h1>Logo</h1>
  <div>Navigation</div>
</nav>
```

12. What is the difference between justify-center and items-center?

Answer: `justify-center` aligns items along the main axis. `items-center` aligns items along the cross axis.
The exact direction depends on `flex-direction`.

13. How do you create a grid with three columns?

Answer:

```
<div class="grid grid-cols-3 gap-4">
  <div>One</div>
  <div>Two</div>
  <div>Three</div>
</div>
```

`grid` enables CSS Grid, `grid-cols-3` creates three columns, and `gap-4` adds spacing.

14. What is the difference between Flexbox and Grid?

Answer: Flexbox is primarily one-dimensional, arranging elements in a row or column. Grid is two-dimensional, allowing control over rows and columns together.

15. How do you center an element horizontally and vertically?

Answer: One common method is Flexbox:

```
<div class="flex min-h-screen items-center justify-center">
  <div class="rounded-lg bg-blue-500 p-6 text-white">
    Centered Content
  </div>
</div>
```

Section 4: Responsive Design

16. How does responsive design work in Tailwind CSS?

Answer: Tailwind uses breakpoint variants to apply styles at specified viewport widths.

```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3">
  <!-- Cards -->
</div>
```

This displays one column by default, two at the `md` breakpoint, and three at `lg`.

17. What are the default responsive breakpoints?

Answer: The standard default breakpoints are:

Prefix	Minimum width
`sm:`	640px
`md:`	768px
`lg:`	1024px
`xl:`	1280px
`2xl:`	1536px

These can be customized through the theme.

18. What is mobile-first design?

Answer: Mobile-first design means writing the base styles for smaller screens first and adding breakpoint-specific styles for larger screens.

```
<h1 class="text-2xl md:text-4xl lg:text-6xl">
  Responsive Heading
</h1>
```

19. How do you hide an element on mobile and show it on desktop?

Answer:

```
<div class="hidden md:block">
  Desktop Content
</div>
```

`hidden` hides the element by default, and `md:block` displays it at the medium breakpoint and above.

20. Can Tailwind CSS use custom breakpoints?

Answer: Yes. In Tailwind CSS v4, define a custom breakpoint using `@theme`.

```
@import "tailwindcss";

@theme {
  --breakpoint-3xl: 120rem;
}
```

You can then use a variant such as `3xl:text-5xl`.

Section 5: Backgrounds and Visual Effects

21. How do you create a gradient background?

Answer: Use gradient utilities such as `bg-linear-to-r`, `from-*`, `via-*`, and `to-*` in Tailwind CSS v4.

```
<div class="bg-linear-to-r from-blue-500 to-purple-600 p-8 text-white">
  Gradient Background
</div>
```

22. How do you set a background image?

Answer: Use an arbitrary background-image utility or define a reusable background in CSS.

```
<div class="bg-[url('/images/hero.jpg')] bg-cover bg-center">
  Hero Section
</div>
```

23. What are opacity and backdrop blur?

Answer: Opacity controls the transparency of an element. Backdrop blur blurs the content visible behind an element.

```
<div class="bg-white/70 p-6 backdrop-blur-md">
  Glass Effect
</div>
```

24. How do you control an image's size and position?

Answer: Use `object-cover`, `object-contain`, `object-center`, and other `object-*` utilities.

```
<img
  src="/images/photo.jpg"
  alt="Landscape"
  class="h-64 w-full rounded-xl object-cover object-center"
/>
```

25. How do you create a card with a shadow and rounded corners?

Answer:

```
<div class="rounded-2xl border border-gray-200 bg-white p-6 shadow-xl">
  <h2 class="text-xl font-bold">Product Card</h2>
  <p class="mt-2 text-gray-600">Product description.</p>
</div>
```

Section 6: Typography and Text

26. How do you style text in Tailwind CSS?

Answer: Use typography utilities such as `text-*`, `font-*`, `leading-*`, `tracking-*`, and text alignment utilities.

```
<p class="text-lg font-medium leading-7 tracking-wide text-gray-700">
  Readable paragraph text.
</p>
```

27. What is the difference between text-sm, text-base, and text-xl?

Answer: These classes represent different font sizes. In the default theme, `text-sm` is smaller than `text-base`, while `text-xl` is larger. Exact values can be customized.

28. How do you truncate overflowing text?

Answer: Use `truncate` to clip overflowing text and show an ellipsis on a single line.

```
<p class="w-48 truncate">
  This is a long paragraph that will be truncated.
</p>
```

For multiple lines, use a line-clamp utility such as `line-clamp-2`.

29. How do you change font weight and line height?

Answer: Use `font-*` for font weight and `leading-*` for line height.

```
<p class="font-bold leading-8">
  This is a bold paragraph with increased line height.
</p>
```

30. How do you use a custom font?

Answer: Load the font through a stylesheet or font provider, then register the font family in the Tailwind theme.

```
@import "tailwindcss";

@theme {
```

```
  --font-display: "Poppins", sans-serif;
}
```

Use it with `font-display`.

Section 7: States and Variants

31. What are variants in Tailwind CSS?

Answer: Variants modify when or under which conditions a utility applies. Examples include `hover:`, `focus:`, `active:`, `disabled:`, `dark:`, and responsive variants such as `md:`.

32. How do you create a hover effect?

Answer:

```
<button class="rounded-lg bg-blue-600 px-5 py-3 text-white hover:bg-blue-800">
  Click Me
</button>
```

The background changes when the user hovers over the button.

33. What is the difference between hover: and focus:?

Answer: `hover:` applies when the pointer hovers over an element. `focus:` applies when the element receives focus, such as when a user selects an input or navigates to a button using the keyboard.

34. What are group-hover: and peer-checked:?

Answer: `group-hover:` allows a child element to respond when its parent with the `group` class is hovered. `peer-checked:` allows a later sibling to respond when a peer input is checked.

```
<label class="group flex cursor-pointer items-center gap-3">
  <input type="checkbox" class="peer" />
  <span class="group-hover:text-blue-600 peer-checked:font-bold">
    Select this option
  </span>
</label>
```

35. How do you implement dark mode?

Answer: Use `dark:` variants and configure the theme strategy according to the project requirements.

```
<div class="bg-white p-6 text-black dark:bg-gray-900 dark:text-white">
  Theme-aware content
</div>
```

The dark-mode strategy determines whether the styles follow system preference or a manually controlled theme.

Section 8: Transitions and Animations

36. What is the difference between transitions and animations?

Answer: A transition smoothly changes a property between states, such as a button's background color on hover. An animation can use keyframes to define multiple stages and can run automatically or repeatedly.

37. What do duration-*, delay-*, and ease-* do?

Answer:

- `duration-300` sets a 300ms transition or animation duration.

- `delay-150` adds a 150ms delay.
- `ease-in-out` controls the timing curve.

```
<button class="transition-colors duration-300 ease-in-out hover:bg-green-600">
  Hover Me
</button>
```

38. What are Tailwind's built-in animations?

Answer: Tailwind includes `animate-spin`, `animate-ping`, `animate-pulse`, and `animate-bounce`. These are commonly used for spinners, status indicators, skeleton loaders, and attention cues.

39. How do you apply transform effects?

Answer: Use utilities such as `translate-*`, `rotate-*`, `scale-*`, and `skew-*`.

```
<div class="transition-transform duration-300 hover:-translate-y-2 hover:scale-105">
  Hover Card
</div>
```

40. How do you create custom animations and support reduced motion?

Answer: Define custom keyframes and an animation token in the theme. Use reduced-motion variants to respect accessibility preferences.

```
<div class="animate-spin motion-reduce:animate-none">
  Loading
</div>
```

In Tailwind CSS v4, custom animations can be defined using `@theme` and `@keyframes`.

Section 9: Advanced Tailwind CSS

41. What are arbitrary values and arbitrary properties?

Answer: Arbitrary values allow custom values inside square brackets. Arbitrary properties allow CSS declarations without requiring a predefined utility.

```
<div class="w-[350px] [mask-type:luminance]">
  Custom Styling
</div>
```

42. What is the @theme directive?

Answer: In Tailwind CSS v4, `@theme` defines theme variables that can generate corresponding utility classes and variants.

```
@theme {
  --color-brand: #16a34a;
  --radius-card: 1.25rem;
}
```

This can provide utilities such as `bg-brand` and `rounded-card`.

43. What are custom utilities and plugins?

Answer: Custom utilities add project-specific styling classes. Plugins can extend Tailwind with utilities, variants, or other integrations. In v4, `@utility` is useful for defining many custom utilities; plugin

approaches also depend on the project setup.

44. How does Tailwind detect classes, and what are design tokens?

Answer: Tailwind scans source files to identify class names and generates the corresponding CSS. Dynamically constructed class names can be missed, so complete class names should be present in detectable source code. Design tokens are reusable values for colors, spacing, fonts, and other design decisions.

45. How do you optimize and debug a Tailwind production build?

Answer: Check source detection, CSS imports, class spelling, responsive variants, and CSS overrides. Use production builds to verify generated styles, avoid unnecessary JavaScript and animations, optimize images, and use browser DevTools to inspect computed styles.

Section 10: Forms and Interactivity

46. How do you style inputs, textareas, and select elements?

Answer: Use utilities for borders, padding, backgrounds, rounded corners, and focus states.

```
<input class="rounded-lg border p-3 focus:border-blue-500 focus:ring-2" />
```

Similar utilities can be applied to `textarea` and `select` elements.

47. How do you style placeholders and file inputs?

Answer: Use the `placeholder:` variant to style placeholder text and the `file:` variant to style the file upload button.

```
<input
  type="file"
  class="file:rounded-md file:bg-blue-600 file:px-4 file:py-2 file:text-white"
/>
```

48. How do you style form validation states?

Answer: Use state variants such as `required:`, `valid:`, `invalid:`, and `disabled:`. These style the current state but do not replace actual validation logic.

```
<input
  type="email"
  required
  class="border p-3 invalid:border-red-500 valid:border-green-500"
/>
```

49. What are accent-*, caret-*, pointer-events-*, and touch-action utilities?

Answer:

- `accent-*` styles the accent color of form controls.
- `caret-*` changes the text cursor color.
- `pointer-events-*` controls pointer-event handling.
- `touch-*` utilities control touch gestures.

Example: `accent-blue-600` `caret-red-500` `pointer-events-auto` `touch-pan-y`.

50. How do you create accessible dropdowns, sticky headers, scrolling layouts, and modals?

Answer: Use `sticky top-0` for sticky headers, `snap-x` and `scroll-smooth` for scrolling layouts, and an appropriate HTML structure for dropdowns. Native `<dialog>` with `showModal()` is a useful way to

implement modal behavior. Ensure keyboard access, visible focus indicators, appropriate labels, and correct focus management.

CHAPTER 13

Mini Projects — Combine Your Skills

Nelaa skills-um ore project-la combine panni practice pannunga.

Idhu varaikkum kathukkiya ellaa concepts-um (layout, responsive, typography, states, animations, forms) real project-la eppadi use pandradhu-nu paakka indha 4 mini projects. Ovvoru project-um difficulty level-padi arrange pannirukken.

Project 1: Developer Portfolio (Intermediate)

Build a responsive portfolio with About, Skills, Projects, and Contact sections. Include dark mode, hover effects, and reusable cards.

- **Sections:** Hero, About, Skills, Projects, Contact.
- **Skills:** responsive navbar, Grid layout, `group-hover:` effects, dark mode toggle.
- **Bonus:** Reusable project card component create pannunga.

Project 2: E-commerce UI (Intermediate)

Create a product grid, category filters, product cards, a responsive navbar, and a cart summary layout.

- **Sections:** Navbar, category filter bar, product grid, cart summary panel.
- **Skills:** `grid-cols-1 sm:grid-cols-2 lg:grid-cols-4`, badges, price typography, hover zoom images.
- **Bonus:** Sticky cart summary for desktop layout.

Project 3: Admin Dashboard (Advanced)

Build a sidebar, analytics cards, a data table, responsive grid layouts, and accessible dropdowns.

- **Sections:** Sidebar, topbar, analytics cards, data table, activity feed.
- **Skills:** responsive sidebar behavior, `snap-*` scrolling tables, `sticky` headers, `<details>` dropdowns.
- **Bonus:** Dark mode support for the entire dashboard.

Project 4: Form Validation UI (Advanced)

Create a complete registration form with validation states, file uploads, custom checkboxes, and an accessible success dialog.

- **Sections:** Personal details, account credentials, profile image upload, terms checkbox, submit flow.
- **Skills:** `invalid:` / `valid:` states, `file:` styling, `accent-*`, native `<dialog>` success modal.
- **Bonus:** `focus-visible:` outlines add panni keyboard accessibility test pannunga.

FINAL ADVICE

Ovvoru project-um mudinja piragu, browser DevTools-la mobile, tablet, desktop view-la test pannunga. Console-la build errors irundhaal fix pannunga. Plus, unga project-ai GitHub-la host panni portfolio-kulla add pannunga — interview-la kaattave ready!

Project Checklist

Checklist Item	Why It Matters
Responsive on mobile, tablet, desktop	Real users use different devices

Checklist Item	Why It Matters
Keyboard navigation works	Accessibility and usability
Focus indicators visible	Keyboard users can track position
Images have alt text	Screen readers and SEO
Contrast sufficient	Readability for everyone
No console errors	Production cleanliness
Production build tested	Generated CSS includes all classes

Continue Your Learning Journey

- Official docs: tailwindcss.com/docs
- Practice daily — ovvoru naalum oru chinna component build pannunga.
- Build real projects — portfolio, blog, dashboard, e-commerce UI.
- Teach someone — padikkaradha vida solikodukkarathu dhaan best revision.

REMEMBER

Tailwind CSS enbadhu CSS-ku replacement illa — oru speed tool. Fundamentals strong-ah irundha, endha framework-um easy-ah kathukka mudiyum. Keep building!

Happy Coding! 🚀

Nandri — indha complete Tailwind CSS guide-ai padithatharku. Unga learning journey smooth-ah and successful-ah irukka vazhthukkal.

— Dhamodharan

Tailwind CSS — Complete Learning Guide · 2026 Edition · Author: Dhamodharan.
Tailwind CSS is a trademark of Tailwind Labs Inc. This is an independent educational resource and is not affiliated with Tailwind Labs.
Code examples are provided for learning purposes — always test in your own project setup.